

m-format constellation -- Command-line Examples

mnemonic-toolkit v0.55.3 -- worked examples (Linux), exact verbatim I/O

2026-06-15

Contents

About these examples	2
Seed input from a file	2
Multiple files for multiple seeds	2
1. Install the constellation on Linux	3
2. Single-sig card set from a seed phrase (file input)	5
3. Conventional 2-of-3 multisig from 3 seed phrases (per-device, file input)	6
3.4 Building from all seeds on one machine (multiple files, one command)	8
4. Card set -> Bitcoin Core wallet descriptor (and how to import)	10
5. Custom degrading-miniscript wallet (distinct keys per tier) + watch-only export	12
5.1 The secret word (a hashlock shared by tiers 1 and 2)	12
5.2 The guided builder caps complexity -- use the raw descriptor path	12
5.3 Engrave the card set	15
5.4 Watch-only export for Nunchuk / Core / Sparrow	18
6. Taproot version of the degrading wallet	19
6.1 Why depth-1 (the one-tier-per-leaf limit on master)	19
6.3 Engrave + first address	20
6.5 Adding a condition: which depth? (a spending-cost comparison)	21
Appendix A -- the public test seeds used	23
Appendix B -- EXPERIMENTAL: depth->=2 taproot reconstruction (NOT FOR REAL FUNDS)	24

About these examples

This document shows real, copy-pasteable command lines for the **m-format constellation** -- a steel-engravable Bitcoin backup system built around four CLIs (`mnemonic`, `md`, `ms`, `mk`). Every command below was executed against `mnemonic v0.55.3` on Linux and **both the command and its full output are reproduced verbatim** -- no abbreviations, no ellipses, no elided keys or addresses. Long lines wrap with a grey hook-arrow continuation marker in the left margin.

All seed phrases here are public BIP-39 TEST VECTORS (`abandon abandon ... about`, etc.). They are world-known and hold no funds. **Never type a real seed phrase onto a networked machine, and never reuse these test phrases for real money.** The toolkit is alpha software -- use only with disposable amounts or on testnet until it has been independently audited.

Seed input from a file

Yes -- the toolkit reads a seed phrase from a file. There is no `--phrase-file` flag; instead every secret slot accepts the value `-`, meaning **"read this secret from stdin"**, which you point at a file with the shell's `< file` redirect:

```
$ mnemonic bundle --template bip84 --network mainnet --slot @0.phrase=- < seed.txt
```

This is the **secure** idiom: the phrase travels on stdin, so it never appears in `argv` / `/proc/$PID/cmdline` or your shell history (unlike the inline form `--slot @0.phrase='<words>'`, which the toolkit flags as `secret-on-argv`). The reader strips a single trailing newline, so an ordinary one-line text file works.

Multiple files for multiple seeds

Only **one** stdin secret is allowed per invocation, so `< file` reads exactly one seed. To use several seed files there are two patterns:

1. **One file per invocation (secure -- recommended).** Process each seed file in its own command and combine only the resulting public xpubs. No machine ever holds more than one seed, and nothing secret reaches `argv`. This is the per-device 2-of-3 multisig flow in section 3.
2. **Several files in one command (convenient -- less safe).** Read each file with command substitution, `--slot "@N.phrase=$(cat seedN.txt)"`. This works for any number of seeds, but each substituted phrase lands on `argv` (the toolkit prints a `secret-on-argv` warning). Only one slot may use the `-` stdin form; a second `-` is rejected. Shown in section 3.4.

Convention in this document: whenever a command reads a file (a seed, a descriptor, a policy JSON, or an `md1` chunk list), the file's contents are printed with `cat` immediately beforehand, so every input is visible.

The three engraved cards:

Card	What it carries
ms1	BIP-39 entropy (recovers the seed)
mk1	xpub + origin (master fingerprint + BIP path)
md1	wallet policy (descriptor template + bound xpubs)

Throughout, `$` is the shell prompt; everything after it is what you type.

1. Install the constellation on Linux

The in-repo installer builds each component with `cargo install --locked` into `~/cargo/bin` (no `sudo`, no system files touched). It needs `cargo`, `git`, and a C toolchain; the CLIs require `rustc >= 1.85`.

Install all four CLIs (this compiles from source, so the build log is machine-specific and not reproduced here):

```
$ sh -c "$(curl -fsSL https://raw.githubusercontent.com/bg002h/mnemonic-toolkit/master/scripts/install.sh)" -- --no-gui
```

The installer carries the current version pins, so it never goes stale. Useful flags: `--only <c>`, `--exclude <c>`, `--no-gui`, `--from-git`, `--force`, `--dry-run`, `--list`. The pin table (`--list`) and a dry run are deterministic:

```
$ sh '/scratch/code/shibboleth/mnemonic-toolkit/scripts/install.sh' --list
COMPONENT      CARGO_PACKAGE  DEFAULT  FEATURES  GIT_TAG
-----
mnemonic       mnemonic-toolkit  git (only)  (none)    mnemonic-toolkit-v0.55.3
md             md-cli          crates.io  cli-compiler  descriptor-mnemonic-md-cli-v0.6.2
ms            ms-cli          crates.io  (none)     ms-cli-v0.7.0
mk            mk-cli          crates.io  (none)     mk-cli-v0.8.0
mnemonic-gui   mnemonic-gui     git (only)  (none)     mnemonic-gui-v0.40.0
```

```
$ sh '/scratch/code/shibboleth/mnemonic-toolkit/scripts/install.sh' --no-gui --dry-run
m-format constellation installer
install root: /home/bcg/.cargo/bin
source: crates.io (default; mnemonic-toolkit stays on git+tag)

install mnemonic (git: mnemonic-toolkit-v0.55.3)
[dry-run] cargo install --locked --git https://github.com/bg002h/mnemonic-toolkit --tag mnemonic-toolkit-v0.55.3 mnemonic-toolkit
install md (crates.io: md-cli)
[dry-run] cargo install --locked --features cli-compiler md-cli
install ms (crates.io: ms-cli)
[dry-run] cargo install --locked ms-cli
install mk (crates.io: mk-cli)
[dry-run] cargo install --locked mk-cli
skip mnemonic-gui

4 installed.

verify:
  mnemonic --version      md --version
  ms --version            mk --version
  mnemonic-gui --version
```

Verify the install and list every subcommand:

```
$ mnemonic --version
mnemonic 0.55.3
```

```
$ mnemonic --help
engraving-bundle CLI for the m-format star (msl + mkl + mdl)

Usage: mnemonic [OPTIONS] <COMMAND>

Commands:
  bundle          emit a 3-card engraving bundle from a phrase or xpub
  verify-bundle   round-trip-check an engraved bundle
  convert         convert between seed/key formats (BIP-39 / BIP-32 / WIF / msl / mkl)
  addresses       list a wallet's receive/change addresses (batch, read-only)
  decode-address  decode a Bitcoin address → network(s) / script type / witness version / scriptPubKey
  export-wallet   emit watch-only wallet artifacts (Bitcoin Core importdescriptors, BIP-388 wallet_policy)
  import-wallet   import a third-party wallet blob into an m-format bundle (v0.26.0 Phase 2: BSMS Round-2 only)
  derive-child    derive deterministic child entropy / keys from a master xprv (BIP-85)
  electrum-decrypt decrypt an Electrum field-encrypted secret (seed phrase / xprv) with a password
  final-word      emit the set of BIP-39 last words that yield a valid checksum for an N-1 partial phrase
  seed-xor       split a BIP-39 phrase into N XOR shares OR combine N shares back into a phrase
  seedqr         encode/decode SeedQR (BIP-39 mnemonic ↔ numeric digit-string QR payload)
  nostr           Wrap an existing nostr key (npub/nsec) as Bitcoin addresses/descriptors/WIF
  silent-payment  Derive a BIP-352 silent-payment receiver address (base + labeled) from a seed
  slip39         split a master secret into SLIP-39 K-of-N shares OR combine shares back (Trezor-compatible)
  ms-shares      split a secret into BIP-93 codex32 K-of-N (msl) shares OR combine shares back
  gui-schema     emit SPEC §7 GUI-overlay flag-surface schema JSON (companion to `mnemonic-gui` v0.2)
  repair         BCH error-correct a corrupted m-format card (msl / mkl / mdl)
  inspect        describe the contents of an m-format card (msl / mkl / mdl)
  compare-cost   compare wsh-vs-tr per-spending-condition cost for a miniscript or descriptor
  xpub-search    search for a target (xpub, descriptor, address, or passphrase) under a seed or xpub
  verify-message verify a Bitcoin message signature (legacy P2PKH signmessage + BIP-322 segwit/taproot)
  restore        emit a watch-only restore document (single-sig) from a seed + optional passphrase
  build-descriptor build a validated wsh(...) descriptor + BIP-388 policy from a JSON policy-tree spec
  help           Print this message or the help of the given subcommand(s)
```

Options:

```
--no-auto-repair v0.22.0 – skip auto-fire repair on decode failures; preserve pre-v0.22 exit policy. Global flag. Honored by `convert`, `inspect`,  
↳ and (v0.22.1+) `verify-bundle`. For `verify-bundle`, auto-fire is additionally gated on `std::io::stdout().is_terminal()` to preserve the  
↳ legacy VerifyCheck-row behavior when output is piped or captured (per v0.22.1 D18 – TTY-conditional default). Standalone `repair` ignores this  
↳ flag (the whole point of that subcommand IS repair). Under `--json` calling contexts the auto-fire emits a structured JSON envelope on stdout (  
↳ per v0.22.1 D20) instead of text-form  
-h, --help          Print help  
-V, --version       Print version
```

RECOVERING A FORGOTTEN BIP-39 PASSPHRASE:

If you have your seed words (entropy) but not the BIP-39 passphrase (the optional "25th word"): if you have a LIST of likely passphrases, `mnemonic xpub-search passphrase-of-xpub --passphrase-candidates-file <file> --target-xpub <a known xpub>` tests each candidate against a value you already know. To GENERATE or mutate a keyspace (wordlists, masks, typo models), `mnemonic` does not – an external open-source tool does: `btcrecover` searches passphrase candidates and confirms each by deriving an address / xpub / master-fingerprint at common default paths and matching a value you already know.

`btcrecover` (maintained): <https://github.com/3rdIteration/btcrecover>
original: <https://github.com/gurnec/btcrecover>

Pointer current as of 2026-05-25. Run untrusted recovery tools offline, on an air-gapped machine.

2. Single-sig card set from a seed phrase (file input)

Create a native-segwit (BIP-84, $m/84'/0'/0'$) single-sig 3-card bundle from one seed phrase held in a file. Write the phrase to `seed0.txt` (here a public test vector) and feed it on stdin:

```
$ printf '%s\n' 'abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about' > seed0.txt
```

```
$ cat seed0.txt
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about
```

(`--template` choices for single-sig: `bip44`, `bip49`, `bip84`, `bip86`.) Run the bundle. `stdout` carries the three cards to engrave; `stderr` carries the human-readable engraving panel and the secret-material warning:

```
$ mnemonic bundle --template bip84 --network mainnet --slot @0.phrase=- < seed0.txt
# ms1 (entropy, BCH-checksummed)
ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqqc9sraq34v7f

ms10e ntrsq qqqqq qqqqq qqqqq qqqqq qqqc9 sraq 34v7f

# mk1 (xpub + origin)
mk1qprsqhqqq3cqt3leutks2qvz3vs70mejkhk622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8qp3yxg3dpe854wq4
mk1qprsqhqp0f30mtxzd65mvwcur9usdatwuvq6z70r9nwrgrk6xn6l8gy6nwa2n977sw6zh34rma0nh

mk1qp rsqhp qqsq3 cqt3l eutk s2qvz g3vs7 0mej h k622w s2kgd
emj2c d8zwj 2skzx 2wq0q w70l4 q99vd yh5x0 z8v4y slsp8 qp3yx
g3dpe 854wq 4
mk1qp rsqhp p0f30 mtxzd 65mvw cur9u sdatw uvq6 z70r9 nwrgrk
6xn6l 8gy6n wa2n9 77sw6 zh34r ma0nh

# md1 (wallet policy)
md1fgdxlpqpqm6jzzqqvqpdw0za5zs4gyy55aq4vsmnh4s6wyaypu34c7raqu8np
md1fgdxlpqf2zcgefcpumel75q5435j7seugaj5jr7qyur6vt76es5cdeyrq7zdy0d
md1fgdxlpq3xa2dk8vwpj7gx74hwqxqdp083jehp5tdrf0n5zdfkqcdlrvnh5r62jn

md1fg-dxlpq-pqpm6-jzzqq-vqpdq-w0za5-zs4gy-y55aq-4vsmn-hy4s6-wyayp-u34c7-raqu8-np
md1fg-dxlpq-f2zcg-efcpu-pmel7-5q543-5j7se-ugaj5-jr7qy-ur6vt-76es5-cdeyr-q7zdy-0d
md1fg-dxlpq-3xa2d-k8vwp-j7gx7-4hwqx-qdp08-3jehp-5tdrf-a0n5z-dfkqc-dlrvn-h5r62-jn

# === Wallet bundle: bip84, mainnet ===
# ms1: 1c017
# mk1: 1c017
# fingerprint: 73c5da0a
# origin path: m/84'/0'/0'
# Template: bip84
# md1: 1c01
warning: stdout carries private key material (can spend) – redirect or encrypt (e.g. '> file.txt' or '| age -e ...')
```

Each card is printed twice: once unbroken, once grouped into 5-character blocks (`ms10e ntrsq qqqqq ...`) -- the grouped form is what you punch or engrave. Add `--no-engraving-card` to suppress the `stderr` panel when piping into other tools.

3. Conventional 2-of-3 multisig from 3 seed phrases (per-device, file input)

A real multisig never lets one machine see more than one seed. Each cosigner derives **only their public xpub** from their own seed file (on their own, ideally air-gapped, device); the coordinator then combines the three **public** keys into a watch-only `wsh(sortedmulti(...))`. No secret ever leaves its file.

Put each cosigner's seed in its own file:

```
$ printf '%s\n' 'abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about' > seed0.txt
```

```
$ printf '%s\n' 'legal winner thank year wave sausage worth useful legal winner thank yellow' > seed1.txt
```

```
$ printf '%s\n' 'letter advice cage absurd amount doctor acoustic avoid letter advice cage above' > seed2.txt
```

On each device, derive that cosigner's BIP-87 multisig fingerprint and account xpub (`--template wsh-sortedmulti` implies the `m/87'/0'/0'` path) from the seed file. Cosigner @0:

```
$ cat seed0.txt
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about
```

```
$ mnemonic convert --from phrase=- --to fingerprint --template wsh-sortedmulti --network mainnet < seed0.txt
fingerprint: 73c5da0a
```

```
$ mnemonic convert --from phrase=- --to xpub --template wsh-sortedmulti --network mainnet < seed0.txt
xpub: xpub6DBjYnc4ewKti13Q1L35bqdodw5z3VGJnf516B3icHrEGEUcCuCG5GVQDZtH8Xmsyt3Fs9YDNwLaqjUbbRidwXZ6sxufZcr4VqqzrXvicM
note: stdout is watch-only – public keys only, cannot spend
```

Cosigner @1:

```
$ cat seed1.txt
legal winner thank year wave sausage worth useful legal winner thank yellow
```

```
$ mnemonic convert --from phrase=- --to fingerprint --template wsh-sortedmulti --network mainnet < seed1.txt
fingerprint: b8688df1
```

```
$ mnemonic convert --from phrase=- --to xpub --template wsh-sortedmulti --network mainnet < seed1.txt
xpub: xpub6CbhrPzY2z7NcCGCGjLAJLq8iRyjUfwmDXQs66MxTVURekqb9DpLnVJ5D1qpatZjUuPGTyxf5TYU1vA34YFE9FHB4TvfYmokYLVsyEFZFt9
note: stdout is watch-only – public keys only, cannot spend
```

Cosigner @2:

```
$ cat seed2.txt
letter advice cage absurd amount doctor acoustic avoid letter advice cage above
```

```
$ mnemonic convert --from phrase=- --to fingerprint --template wsh-sortedmulti --network mainnet < seed2.txt
fingerprint: 28645006
```

```
$ mnemonic convert --from phrase=- --to xpub --template wsh-sortedmulti --network mainnet < seed2.txt
xpub: xpub6DB7HNqW6CZojxN85NuFTPWZhi2FagSnexPS1rv3nYQhngkmdHgb7iebYvTFmFKKDA3ozf5yezDsCH6cXAw3WZijviSZtZC2hjHn2uazz4z
note: stdout is watch-only – public keys only, cannot spend
```

Wrap each as an origin-annotated descriptor key `[fingerprint/87'/0'/0']xpub` and combine into a 2-of-3 sorted-multisig descriptor (`/<0;1>/*` = the external/change multipath). The assembled descriptor file:

```
$ cat multisig.desc
wsh(sortedmulti(2,[73c5da0a/87'/0'/0']xpub6DBjYnc4ewKti13Q1L35bqdodw5z3VGJnf516B3icHrEGEUcCuCG5GVQDZtH8Xmsyt3Fs9YDNwLaqjUbbRidwXZ6sxufZcr4VqqzrXvicM
↳ /<0;1>/*,[b8688df1/87'/0'/0']xpub6CbhrPzY2z7NcCGCGjLAJLq8iRyjUfwmDXQs66MxTVURekqb9DpLnVJ5D1qpatZjUuPGTyxf5TYU1vA34YFE9FHB4TvfYmokYLVsyEFZFt9
↳ /<0;1>/*,[28645006/87'/0'/0']xpub6DB7HNqW6CZojxN85NuFTPWZhi2FagSnexPS1rv3nYQhngkmdHgb7iebYvTFmFKKDA3ozf5yezDsCH6cXAw3WZijviSZtZC2hjHn2uazz4z
↳ /<0;1>/*))
```

Validate and canonicalise it (this also computes the BIP-380 checksum):

```
$ mnemonic export-wallet --descriptor "$(cat multisig.desc)" --format descriptor --network mainnet
wsh(sortedmulti(2,[73c5da0a/87'/0'/0']xpub6DBj1Ync4ewKti13Q1L35bqdodw5z3VGJnf516B3icHrEGEUcCuG5GVQDZtH8Xmsyt3Fs9YDNwLaqjUbbRidwXZ6sxufZcr4VqqzrXvicM
↳ /<0;l> /*, [b8688df1/87'/0'/0']xpub6CBhrPzY2z7NcCGCgJLAJLq8iRyJUfwmdXQs66MxTVURekqb9DpLnVJ5D1qpatZJUuPGTyxf5TYU1vA34YFE9FHB4TvfYmokYLVsyEFZFt9
↳ /<0;l> /*, [28645006/87'/0'/0']xpub6DB7HNqw6CZojxN85NuFTPWZhi2FagSnexPS1rv3nYQhngkmdHgb7iebYvTFmFKKA3ozf5yezDsCH6cXAw3WZijviSZtZC2hjHn2uazz4z
↳ /<0;l> /*))#4wup4at0
note: stdout is watch-only – public keys only, cannot spend
```

The first receive address (here via the BSMS / BIP-129 record, which also carries the /0/*,/1/* derivation):

```
$ mnemonic export-wallet --descriptor "$(cat multisig.desc)" --format bsms --network mainnet
BSMS 1.0
wsh(sortedmulti(2,[73c5da0a/87'/0'/0']xpub6DBj1Ync4ewKti13Q1L35bqdodw5z3VGJnf516B3icHrEGEUcCuG5GVQDZtH8Xmsyt3Fs9YDNwLaqjUbbRidwXZ6sxufZcr4VqqzrXvicM
↳ /<0;l> /*, [b8688df1/87'/0'/0']xpub6CBhrPzY2z7NcCGCgJLAJLq8iRyJUfwmdXQs66MxTVURekqb9DpLnVJ5D1qpatZJUuPGTyxf5TYU1vA34YFE9FHB4TvfYmokYLVsyEFZFt9
↳ /<0;l> /*, [28645006/87'/0'/0']xpub6DB7HNqw6CZojxN85NuFTPWZhi2FagSnexPS1rv3nYQhngkmdHgb7iebYvTFmFKKA3ozf5yezDsCH6cXAw3WZijviSZtZC2hjHn2uazz4z
↳ /<0;l> /*))#4wup4at0
/0/*,/1/*
bc1qksssl2m6t3aynza394sr9m86vt6md2v76kj52jun2xlrdeaa4q84qtpl
note: stdout is watch-only – public keys only, cannot spend
```

Engrave the shared watch-only card set from the public descriptor (the md1 policy card is shared by all cosigners; each cosigner additionally backs up their own seed as a single-sig ms1 set per section 2). With only public xpubs supplied, the ms1 cards are empty placeholders:

```
$ mnemonic bundle --descriptor-file multisig.desc --network mainnet
# ms1 (omitted – descriptor watch-only mode)

# mk1[0] (cosigner 0 xpub + origin)
mk1qperpupqqspsu3s7denyv8nwverpumnrrnchd5pcy3zepa59349dcgs5n5stvpvk3v8x4eqsdngy2jl4wddt5ac2ptv4fya76tpreapyrdfrq
mk1qperpuppnq8kp6xcpphr7svxkwx5ag99s9zfyml9v7tcqrexmpdj7jgqgmny8rr0z7vlj7eqzv2486xkkcfrzq8

mk1qp erpup qqspsu 3s7de nyv8n wverp umnrrn chd5 pcy3z epa59
349dc gs5n5 stvpv k3v8x 4eqsd ngy2j l4wdd t5ac2 ptv4f ya76t
preap yrdfq r
mk1qp erpup pnq8k p6xcp qphr7 svxwk xx5ag 99s9z fymL9 v7tcq
rexmp dj7jg qgmny 8rr0z 7vlj7 eqzv2 486xk kcftr zq8

# mk1[1] (cosigner 1 xpub + origin)
mk1qperpapqqspsu3s7denyv8nwverpumn9cdxzlzpcy3zepaqxfl630m4as45q6fz4ltsntlues3e3gylcu6jsa6jdz69hy0whcpg2f28j5awu
mk1qperpap02p3qtffjqpxtu9j5jaevqs7jqje40324vxlw0txswawpxte3zmhzp6lrpj3ga2lw65h2dd24cuerkpa

mk1qp erpap qqspsu 3s7de nyv8n wverp umn9c dzxzl pcy3z epaqx
afl63 0m4as 45q6f z4lts ntlue s3e3g ylcu6 jsa6j dz69h y0whc
pg2f2 8j5aw u
mk1qp erpap p02p3 qtffj qpxtu j95jz aevqs 7jqje 40324 vxlfw
0txsw awpxt e3zmh zp6lr pj3ga 2lw65 h2dd2 4cuer kpa

# mk1[2] (cosigner 2 xpub + origin)
mk1qperp7pqqspsu3s7denyv8nwverpumpnpgv3gqvpqcy3zapanecw97sn9g25uukrgxca47at8as54xhd0rkqg57cs78mc8fk6507cs8tugnnpnf
mk1qperp7ppr4e59ef7u5p038j900ye439hsphz506fra5k3eac5kg0n7jhn3kpzpn6n0lygd1lpeq4a75lc2mrrz7f

mk1qp erp7p qqspsu 3s7de nyv8n wverp umnpg v3gqv pcy3z epane
cw97s n9g25 uukrg xca47 at8as 54xhd 0rkqg 57cs7 8mc8f k6507
cs8tn ugnpn f
mk1qp erp7p pr4e5 9ef7u 5p038 j900y e439h sphez 506fr a5k3e
ac5kg 0n7jh n3kpz pna6n 0lygd lpfeq 4a75l c2mrr z7f

# md1 (multisig wallet policy)
md1f5przzspq3m67zzqzvrs3pstucw0za5znwrg3hcc5xg5qxzhs7ygg2f6g9kqktgkrn2spp6tlfzprv6ye
md1f5przzswgyrv6pz5hatnt2a8wzs2m92fzrmqarvqsqm3lgr8trr2w5zjcz3yjdjk0qf2g9tn3u5jtsz
md1f5przzsj7q7fkctvh5jqxuepcmcchn8u30m4as45q6fz4ltsntlues3e3gylcu6jsasrwqtape4mvjxf
md1f5przzsafx3dzmj02p3qtffjqpxtu9j5jaevqs7jqje40324vxlw0txswawpxte3zmspu35hx7lrg2t6
md1f5przz3r3qa03segkpx2s4feevxsd3mta6k0mpf2dw678vdsfa3pu0hswvwhxsh98mjsqpsz8xkj9h16g
md1f5przz3gzl20y277fntzt0qr0j9gl5j8mfdmnm3fvs18a908rvzyr8m4xl7gs9e2t6dyvgv4j

md1f5-przsz-pq3m6-7zzq-qvzrs3-pstuc-w0za5-znwrg-3hcc5-xg5qx-zhs7y-yg2f6-g9kqk-tgkrn-2spp6-tlfzprv6ye
md1f5-przsz-wgyrv-6pz5h-atnt2-a8wzs-2m92f-fsrmq-arvqs-qm3lg-xr8tr-r2w5z-jcz3y-jdljk-0qf2g-9tn3u-5jtsz
md1f5-przsz-j7q7-fkctv-h5jqz-xuepc-cmchn-8u30m-4as45-q6fz4-ltsnt-lues3-e3gyl-cu6js-asrwq-tape4-mvjxf
md1f5-przsz-afx3d-zmj02-p3qtff-fjqp-x-tuj95-jzaev-q57jq-je403-24vxl-fw0tx-swawp-xte3z-mspu3-5hx7l-r2t6g
md1f5-prz3-r3qa0-3segk-px2s4-feevx-sd3mt-a6k0m-pf2dw-678vs-dfa3p-u0hsw-nvwhx-sh98m-jsqps-z8xkj-h9l6g
md1f5-prz3-gz1z0-y277f-ntzt0-qr0j9-gl5j8-mfdmnm-3fvs-18a90-8rvzy-r8m4x-l7gs9-e2t6d-yvgv4j

# === Wallet bundle: descriptor, mainnet ===
# Threshold: 2 of 3
# Cosigners:
# @0: (no ms1; watch-only),mk1:c8c3c (73c5da0a @ m/87'/0'/0')
# @1: (no ms1; watch-only),mk1:c8c3d (b8688df1 @ m/87'/0'/0')
# @2: (no ms1; watch-only),mk1:c8c3e (28645006 @ m/87'/0'/0')
# Template: descriptor
# md1: c8c3
# Recovery: any 2 of 3 signing keys + md1 (template card).
note: stdout is watch-only – public keys only, cannot spend
```

3.4 Building from all seeds on one machine (multiple files, one command)

If instead you hold all the seeds yourself, you can build the whole bundle in a single command, reading each seed from its own file with command substitution. This is less safe -- each substituted phrase lands on `argv`, so the toolkit prints a `secret-on-argv` warning for every one -- but it needs no per-device coordination. The three seed files (shown again for reference):

```
$ cat seed0.txt
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about
```

```
$ cat seed1.txt
legal winner thank year wave sausage worth useful legal winner thank yellow
```

```
$ cat seed2.txt
letter advice cage absurd amount doctor acoustic avoid letter advice cage above
```

Because seeds (not just xpubs) are supplied, this emits the **full secret card set** -- one `ms1` per cosigner -- not the watch-only placeholders of 3.3:

```
$ mnemonic bundle --template wsh-sortedmulti --threshold 2 --network mainnet --slot "@0.phrase=$(cat seed0.txt)" --slot "@1.phrase=$(cat seed1.txt)" --
  ↪ slot "@2.phrase=$(cat seed2.txt)"
warning: secret material on argv (--slot @0.phrase=) - pipe via --slot @0.phrase= to avoid /proc/$PID/cmdline exposure
warning: secret material on argv (--slot @1.phrase=) - pipe via --slot @1.phrase= to avoid /proc/$PID/cmdline exposure
warning: secret material on argv (--slot @2.phrase=) - pipe via --slot @2.phrase= to avoid /proc/$PID/cmdline exposure
# ms1[0] (entropy, BCH-checksummed)
ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqc9sraq34v7f

ms10e ntrsq qqqqq qqqqq qqqqq qqqqq qqcj9 sraq 34v7f

# ms1[1] (entropy, BCH-checksummed)
ms10entrsqplh7lml0alh7lml0alh7lml0als5cclar2zmks6

ms10e ntrsq plh7l ml0al h7lml 0alh7 lml0a ls5cc lar2z mksh6

# ms1[2] (entropy, BCH-checksummed)
ms10entrsqzqgppyqsqgppyqsqzqqlfm7mep84hunu

ms10e ntrsq zqgpy yqszy gpyyq szqpy qyqsq qqlfm 7mep8 4hunu

# mk1[0] (cosigner 0 xpub + origin)
mk1qperpupqspu3s7denyv8nwverpumn9cdzxlzpcy3zepa59349dcs5n5stvpvk3v8x4eqsdngy2jl4wddt5ac2ptv4fya76tpreapyrdfqr
mk1qperpuppnq8kp6xcpqphr7svxwx5ag99s9zfym19v7tcqrexmpdj7jgqgmny8rr0z7vlj7eqzv2486xkkcftrq8

mk1qp erpup qqspu 3s7de nyv8n wverp umnrn chdq5 pcy3z epa59
349dc gs5n5 stvpv k3v8x 4eqsd ngy2j l4wdd t5ac2 ptv4f ya76t
preap yrdfq r
mk1qp erpup pnq8k p6xcp qphr7 svxwk xx5ag 99s9z fym19 v7tcq
rexmp dj7jg qgmny 8rr0z 7vlj7 eqzv2 486xk kcftr zq8

# mk1[1] (cosigner 1 xpub + origin)
mk1qperpupqspu3s7denyv8nwverpumn9cdzxlzpcy3zepaqxfl630m4as45q6fz4ltsntlues3e3gylcu6jsa6jdz69hy0whcpg2f28j5awu
mk1qperpapp02p3qtffjqpxtu95jzaevqs7jqje40324vxlw0txswawpxte3zmhzp6lrpj3ga2lw65h2dd24cuerkpa

mk1qp erpap qqspu 3s7de nyv8n wverp umn9c dzxlz pcy3z epaqx
afl63 0m4as 45q6f z4lts ntlue s3e3g ylcu6 jsa6j dz69h y0whc
pg2f2 8j5aw u
mk1qp erpap p02p3 qtffj qpxtu j95jz aevqs 7jqje 40324 vxlfw
0txsw awpxt e3zmh zp6lr pj3ga 2lw65 h2dd2 4cuer kpa

# mk1[2] (cosigner 2 xpub + origin)
mk1qperp7pqspu3s7denyv8nwverpumpgv3gqvpcy3zeganecw97sn9g25uukrgxca47at8as54xhd0rkgs57cs78mc8fk6507cs8tnugnnpnf
mk1qperp7ppr4e59ef7u5p038j900ye439hsphes506fra5k3eac5kg0n7jhn3kpzpa6n0lygdlpfeq4a75lc2mrrz7f

mk1qp erp7p qqspu 3s7de nyv8n wverp umnpg v3gqv pcy3z epane
cw97s n9g25 uukrg xca47 at8as 54xhd 0rkgs 57cs7 8mc8f k6507
cs8tn ugnpn f
mk1qp erp7p pr4e5 9ef7u 5p038 j900y e439h sphez 506fr a5k3e
ac5kg 0n7jh n3kpz pna6n 0lygd lpfeq 4a75l c2mrr z7f

# md1 (multisig wallet policy)
md1f5przzspq3m67zzqqvzrs3pstucw0za5znwrg3hcc5xg5qxzhs7ygg2f6g9kqktgkrn2spp6tlfzprv6ye
md1f5przzswgyrv6pz5hatnt2a8wzs2m92ffsrmqarvqsqm3lgxr8trr2w5zjcz3yjd1jk0qf2g9tn3u5jtsz
md1f5przzsj7q7fkctvh5jqzuepcmcchn8u30m4as45q6fz4ltsntlues3e3gylcu6jsasrwatqpe4mvjxf
md1f5przzsafx3dzmj02p3qtffjqpxtu95jzaevqs7jqje40324vxlw0txswawpxte3zmspu35hx7lrg2t6
md1f5przz3r3qa03segkpx2s4feevxsd3mta6k0mpf2dw678vdsd3pu0hswvwhxsh98mjsqpsz8xkh9l6g
md1f5przz3gzl20y277fntz0qr0j9gl5j8mfdnm3fvs18a908rvzry8m4xl7gs9e2t6dyvjgv4j

md1f5-przzs-pq3m6-7zzqq-vzrs3-pstuc-w0za5-znwrg-3hcc5-xg5qx-zhs7y-yg2f6-g9kqk-tgkrn-2spp6-tlfzp-rv6ye
md1f5-przzs-wgyrv-6pz5h-atnt2-a8wzs-2m92f-fsrmq-arvqs-qm3lg-xr8tr-r2w5z-jcz3y-jd1jk-0qf2g-9tn3u-5jtsz
md1f5-przzs-j7q7f-fkctv-h5jqz-xuepc-cmchn-8u30m-4as45-q6fz4-ltsnt-lues3-e3gyl-cu6js-asrwq-tqpe4-mvjxf
```

```
mdl1f5-przsz-afx3d-zmj02-p3qtf-fjqpx-tuj95-jzaev-qs7jq-je403-24vxl-fw0tx-swapx-xte3z-mspu3-5hx7l-rg2t6
mdl1f5-przz3-r3qa0-3segk-px2s4-feevx-sd3mt-a6k0m-pf2dw-678vs-dfa3p-u0hsw-nvwhx-sh98m-jsqps-z8xkj-h9l6g
mdl1f5-przz3-gz1z0-y277f-ntzt0-qr0j9-gl5j8-mfdrn-m3fvs-l8a90-8rvzy-r8m4x-l7gs9-e2t6d-yvjgv-4j
```

```
# === Wallet bundle: wsh-sortedmulti, mainnet ===
# Threshold: 2 of 3
# Cosigners:
#   @0: ms1:c8c3c,mkl:c8c3c (73c5da0a @ m/87'/0'/0')
#   @1: ms1:c8c3d,mkl:c8c3d (b8688df1 @ m/87'/0'/0')
#   @2: ms1:c8c3e,mkl:c8c3e (28645006 @ m/87'/0'/0')
# Template: wsh-sortedmulti
# mdl: c8c3
# Recovery: any 2 of 3 signing keys + mdl (template card).
warning: stdout carries private key material (can spend) - redirect or encrypt (e.g. '> file.txt' or '| age -e ...')
```

Only one secret may arrive on stdin, so you cannot replace more than one substitution with the `=` file-redirect form -- a second `=` is rejected:

```
$ mnemonic bundle --template wsh-sortedmulti --threshold 2 --network mainnet --slot @0.phrase=- --slot @1.phrase=- --slot "@2.phrase=$(cat seed2.txt)" <
  ↪ seed0.txt
warning: secret material on argv (--slot @2.phrase=) - pipe via --slot @2.phrase=- to avoid /proc/$PID/cmdline exposure
error: at most one --slot @N.<secret>=- per invocation (single stdin per invocation)
```

4. Card set -> Bitcoin Core wallet descriptor (and how to import)

mnemonic restore --md1 <chunks> reconstructs the watch-only wallet from the **shared md1 card alone** -- no seeds needed. First produce that card from the section-3 wallet (descriptor file shown again) and pull out its md1 chunks:

```
$ cat multisig.desc
wsh(sortedmulti(2,[73c5da0a/87'/0'/0']xpub6DBjJYnc4ewKti13Q1L35bqodow5z3VGJnf516B3icHrEGEUcCuG5GVQDZtH8Xmsyt3Fs9YDNwLaqjUbbRidwXZ6sxufZcr4VqqrXvicM
  ↳ /<0;1>/*,[b8688df1/87'/0'/0']xpub6CbhRzPzY2z7NcCGGjLAJLQ8iRyjUfwmdXQs66MxTVUReKqB9DpLnVJ5D1qpatZjUuPGTyxf5TYUlvA34YFE9FHB4TvfyMokYLVsyEFZFt9
  ↳ /<0;1>/*,[28645006/87'/0'/0']xpub6DB7HNQw6CZoJxN85NuFTPWhi2FagSnexPS1rv3nYQhngkmdHgb7iebYvTfMFKDA3ozf5yezDsCH6cXAw3WZijviSZtZC2hjHn2uazz4z
  ↳ /<0;1>/*))
```

```
$ mnemonic bundle --descriptor-file multisig.desc --network mainnet --json | jq -r ".md1[]" > multisig.md1
note: stdout is watch-only - public keys only, cannot spend
```

```
$ cat multisig.md1
md1f5przzspq3m67zzqzvrs3pstucw0za5znwrg3hcc5xg5qxzhs7y9g2f6g9kqktgkgrn2spp6tlfzprv6ye
md1f5przzswgyrv6pz5hatnt2a8wzs2m92ffsrmqarvqsm3lgxr8trr2w5zjcz3yjdLjk0qf2g9tn3u5jtsz
md1f5przzsj7q7fkctvh5jqzxuepcmcn8u30m4as45q6fz4ltsntlues3e3gylcu6j5asrwtqpe4mvjxf
md1f5przzsafx3dzmj02p3qtfjqpxtuj95jzaevqs7jqje40324vxlfw0txswawphte3zmspu35hx7lrg2t6
md1f5przz3rq03se9kpx2s4feevsd3mta6k0mpf2dw678vdsfa3pu0hswvwhxsh98mjsqps28xkjh9l6g
md1f5przz3gzl0y277fntzt0qr0j9gl5j8mfdnm3fvs18a908rvzyr8m4xl7gs9e2t6dyvjgv4j
```

Restore reconstructs the wallet from exactly those chunks. The default form prints the descriptor and first address (note the address matches section 3 -- same wallet -- while the descriptor *string* differs because the md1 card stores each key as a depth-0 master xpub, an equivalent serialisation):

```
$ mnemonic restore --network mainnet $(sed "s/~/--md1 /" multisig.md1)
2-of-3 multisig restore
CONFIRM: verify each cosigner fingerprint against your records before importing.
descriptor: wsh(sortedmulti(2,[73c5da0a/87'/0'/0']
  ↳ xpub661MyMwAQBcFrooZ2966EcDmVX5MoFXZhuJqXTudvJzw8TBfPQSc5JzX52fvS18oqSdEJXJ4kTGRJ76wPWDUSNJsY5JsgVBQoD6KrbdCLL/<0;1>/*,[b8688df1/87'/0'/0']
  ↳ xpub661MyMwAQBcEnFgxHRLx7i1fnjcBPgc71qy8mVkbGXyukNGMK2XFRbAaCLYEJDUUfNoBxTNa68i5MYhqmREkfHjzgHCUEcvJBhXS5bk4RW/<0;1>/*,[28645006/87'/0'/0']
  ↳ xpub661MyMwAQBcEdy4jr5EtEHQBctfscE6a99DGLr2cW4HnnmBSXDoe3odGzRiw3hcRM5wfKcQmb7s5FjdGrR6SrExXmeopaoY9Lk7tQusDjN/<0;1>/*))#yjp7hj7w
first recv: bc1qkssenl2m6t3aynza394sr9m86vt6md2v76kj52jun2x1wrdeaa4q84qtpl
cosigner @0: 73c5da0a [87'/0'/0'] from md1 (not independently verified)
cosigner @1: b8688df1 [87'/0'/0'] from md1 (not independently verified)
cosigner @2: 28645006 [87'/0'/0'] from md1 (not independently verified)
UNVERIFIED: no --from/--cosigner cross-check supplied; verify each cosigner fingerprint above against your records before importing
note: stdout is watch-only - public keys only, cannot spend
```

Add --format bitcoin-core for a ready-to-import importdescriptors request array (external .../0/* + change .../1/*):

```
$ mnemonic restore --network mainnet $(sed "s/~/--md1 /" multisig.md1) --format bitcoin-core
[
  {
    "active": true,
    "desc": "wsh(sortedmulti(2,[73c5da0a/87'/0'/0']
      ↳ xpub661MyMwAQBcFrooZ2966EcDmVX5MoFXZhuJqXTudvJzw8TBfPQSc5JzX52fvS18oqSdEJXJ4kTGRJ76wPWDUSNJsY5JsgVBQoD6KrbdCLL/0/*,[b8688df1/87'/0'/0']
      ↳ xpub661MyMwAQBcEnFgxHRLx7i1fnjcBPgc71qy8mVkbGXyukNGMK2XFRbAaCLYEJDUUfNoBxTNa68i5MYhqmREkfHjzgHCUEcvJBhXS5bk4RW/0/*,[28645006/87'/0'/0']
      ↳ xpub661MyMwAQBcEdy4jr5EtEHQBctfscE6a99DGLr2cW4HnnmBSXDoe3odGzRiw3hcRM5wfKcQmb7s5FjdGrR6SrExXmeopaoY9Lk7tQusDjN/0/*))#y65a0dtg",
    "internal": false,
    "range": [
      0,
      999
    ],
    "timestamp": 0
  },
  {
    "active": true,
    "desc": "wsh(sortedmulti(2,[73c5da0a/87'/0'/0']
      ↳ xpub661MyMwAQBcFrooZ2966EcDmVX5MoFXZhuJqXTudvJzw8TBfPQSc5JzX52fvS18oqSdEJXJ4kTGRJ76wPWDUSNJsY5JsgVBQoD6KrbdCLL/1/*,[b8688df1/87'/0'/0']
      ↳ xpub661MyMwAQBcEnFgxHRLx7i1fnjcBPgc71qy8mVkbGXyukNGMK2XFRbAaCLYEJDUUfNoBxTNa68i5MYhqmREkfHjzgHCUEcvJBhXS5bk4RW/1/*,[28645006/87'/0'/0']
      ↳ xpub661MyMwAQBcEdy4jr5EtEHQBctfscE6a99DGLr2cW4HnnmBSXDoe3odGzRiw3hcRM5wfKcQmb7s5FjdGrR6SrExXmeopaoY9Lk7tQusDjN/1/*))#k0gfvz2t",
    "internal": true,
    "range": [
      0,
      999
    ],
    "timestamp": 0
  }
]
2-of-3 multisig restore
CONFIRM: verify each cosigner fingerprint against your records before importing the payload above.
descriptor: wsh(sortedmulti(2,[73c5da0a/87'/0'/0']
  ↳ xpub661MyMwAQBcFrooZ2966EcDmVX5MoFXZhuJqXTudvJzw8TBfPQSc5JzX52fvS18oqSdEJXJ4kTGRJ76wPWDUSNJsY5JsgVBQoD6KrbdCLL/<0;1>/*,[b8688df1/87'/0'/0']
  ↳ xpub661MyMwAQBcEnFgxHRLx7i1fnjcBPgc71qy8mVkbGXyukNGMK2XFRbAaCLYEJDUUfNoBxTNa68i5MYhqmREkfHjzgHCUEcvJBhXS5bk4RW/<0;1>/*,[28645006/87'/0'/0']
  ↳ xpub661MyMwAQBcEdy4jr5EtEHQBctfscE6a99DGLr2cW4HnnmBSXDoe3odGzRiw3hcRM5wfKcQmb7s5FjdGrR6SrExXmeopaoY9Lk7tQusDjN/<0;1>/*))#yjp7hj7w
first recv: bc1qkssenl2m6t3aynza394sr9m86vt6md2v76kj52jun2x1wrdeaa4q84qtpl
```

```
cosigner @0: 73c5da0a [87'/0'/0'] from md1 (not independently verified)
cosigner @1: b8688df1 [87'/0'/0'] from md1 (not independently verified)
cosigner @2: 28645006 [87'/0'/0'] from md1 (not independently verified)
UNVERIFIED: no --from/--cosigner cross-check supplied; verify each cosigner fingerprint above against your records before importing
note: stdout is watch-only – public keys only, cannot spend
```

Import into Bitcoin Core: save the array, create a blank descriptor wallet, and load it (these run against your own node, so their output is not shown here):

```
$ mnemonic restore --network mainnet $(sed "s/^/--md1 /" multisig.md1) --format bitcoin-core > wallet.json
$ bitcoin-cli -named createwallet wallet_name="multisig-2of3" disable_private_keys=true blank=true descriptors=true
$ bitcoin-cli -rpcwallet="multisig-2of3" importdescriptors "$(cat wallet.json)"
$ bitcoin-cli -rpcwallet="multisig-2of3" getnewaddress
```

Tips: `--timestamp now` skips the rescan for a fresh wallet (default 0 rescans from genesis); `--range 0,4999` widens the gap limit; `--bitcoin-core-version 24` targets older Core. `restore` also emits `--format descriptor` (the bare `wsh(...)#checksum`) for other wallets.

5. Custom degrading-miniscript wallet (distinct keys per tier) + watch-only export

A four-tier vault. **Each tier uses its own distinct key set (no key reuse)**, deliberately mixing all four Bitcoin timelock kinds:

Tier	Spend condition	Timelock kind
1	3-of-3 (K0,K1,K2) + secret word	absolute height -- after(1000000)
2	2-of-3 (K3,K4,K5) + secret word	absolute time -- after(1893456000)
3	both K6 and K7	relative blocks -- older(65535)
4	any 1 of K8,K9,K10	relative time -- older(4255898)

That is **11 distinct keys** (3+3+2+3). Absolute locks (*after*) count from the chain's height/clock; relative locks (*older*) count from each coin's own confirmation. Encodings:

- after(1000000) -- absolute **block height** 1,000,000 (BIP-65; values below 500,000,000 are heights).
- after(1893456000) -- absolute **Unix time** = 2030-01-01 00:00 UTC (values at/above 500,000,000 are timestamps).
- older(65535) -- relative **blocks**: 65,535 blocks (~455 days). This is the largest safe relative-block lock; older(65536) would be BIP-68 consensus-masked to zero, and the toolkit warns if it sees one.
- older(4255898) -- relative **time**: BIP-68's time flag (bit 22) set, plus 61,594 units x 512 s ~ 365 days.

5.1 The secret word (a hashlock shared by tiers 1 and 2)

Reusing a *hash* across tiers is fine -- it is not a key. The secret word is `opensessame`; the descriptor commits to $H = \text{sha256}(\hookrightarrow \text{sha256}(\text{word}))$ and a spend reveals the 32-byte preimage $X = \text{sha256}(\text{word})$:

```
$ python3 -c "import hashlib; w='opensessame'; X=hashlib.sha256(w).digest(); print('preimage X =', X.hex()); print('hash H      =', hashlib.sha256(X).hexdigest())"
preimage X = cd6f70a4440de96063f8dbd7a4a3bbcff6b993af6c83e08d87b359e0760ca9c6
hash H      = a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad
```

5.2 The guided builder caps complexity -- use the raw descriptor path

`mnemonic build-descriptor` runs a satisfiability + cost preview that it **bounds** for funds-safety. An 11-key, 4-branch policy exceeds that envelope, so the guided builder refuses and points you at the raw `--descriptor` path. The policy-tree spec it reads:

```
$ cat policy.json
{
  "schema_version": 1,
  "wrapper": "wsh",
  "root": {
    "or_i": [
      {
        "and_v": [
          {
            "wrap": {
              "w": "v",
              "sub": {
                "after": 1000000
              }
            }
          },
          {
            "and_v": [
              {
                "wrap": {
                  "w": "v",
                  "sub": {
                    "sha256": "a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad"
                  }
                }
              },
              {
                "multi": {
                  "k": 3,
                  "keys": [
                    "[73c5da0a/84'/0'/0']xpub6CatWdiZiodmUeTdp8LT5or8nmbKNcuYvz7WYksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshWcMLoP2fMFMKHPJ4ZeZXYVUhlV1VMrjPC7PW6V"
                  ]
                }
              }
            ]
          }
        ]
      }
    ]
  }
}
```

```

    " [73c5da0a/84'/0'/1']xpub6CatWdiZiodmYVtWLTtEQsAg1H9ooS1bmsJUBwQ83FE1Fyk386FWcyicJgEZv3quZSJKA5dh5Lo2PbubMGxCfZtRthV6ST2qquL9w3HSzcUn
    ↪ ",
    " [73c5da0a/84'/0'/2']xpub6CatWdiZiodmbNGqcQxxjGN165QxTU4PwNNi9WrijYgYf7VxcmuFxsRw3foczLgRDbjDjJbqZhPCTfkcaWmL9BuSw98ybKKJtcHgWeryy6"
  ]
}
]
}
}
]
},
{
  "or_i": [
    {
      "and_v": [
        {
          "wrap": {
            "w": "v",
            "sub": {
              "after": 1893456000
            }
          }
        },
        {
          "and_v": [
            {
              "wrap": {
                "w": "v",
                "sub": {
                  "sha256": "a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad"
                }
              }
            },
            {
              "multi": {
                "k": 2,
                "keys": [
                  "[73c5da0a/84'/0'/3']
                  ↪ xpub6CatWdiZiodmdHurRokjbycCrxddTDJgTsyEAaQfKjKwbwUi79LAWG5gHjMCQB7BeJc47MkubXuZdf45JZHk1qcr1GZ5EwREUDVdLVdPKEC",
                  "[b8688df1/84'/0'/0']
                  ↪ xpub6DNfJehqF1LLUs9kwaqDu12Ajpz9psYVtbGhTykQo1CYdkkqV2vAyR4DiWXSTTDuJwHzVy1AtV6ENGKwgbLWqa4wXMZR4ZmdpRjQB65EgTV",
                  "[b8688df1/84'/0'/1']
                  ↪ xpub6DNfJehqF1LLUsoh1a4XLG3yuFY1oCRJd6Bkba97xvW6SwvY71Nc6LvqqFDLWuyCivJ4eMDxgFtiTxnazgp8rYVzQSKR3L8EGRs9asBarqpe"
                ]
              }
            }
          ]
        }
      ]
    },
    {
      "or_i": [
        {
          "and_v": [
            {
              "wrap": {
                "w": "v",
                "sub": {
                  "older": 65535
                }
              }
            },
            {
              "multi": {
                "k": 2,
                "keys": [
                  "[b8688df1/84'/0'/2']
                  ↪ xpub6DNfJehqF1LLUwZ1DrFcQN365CDMPLSsrqpSwaz4eRrtYz9qWdsVH9JqsHqQ6yNGdG4VXGKbLgcxKkXtwtB5B4iCuHcmjomqR8z6NNxNpU51",
                  "[b8688df1/84'/0'/3']
                  ↪ xpub6DNfJehqF1LLUxb8gRSstbR9LcAxWgwD4V678z5FZ7BLftWzwQCt3yZb5eW4U8AVJwbKcZ2ieggjvKv2xggJpMBRkk3CE7bz7g4uMV7Qp3TU"
                ]
              }
            }
          ]
        },
        {
          "and_v": [
            {
              "wrap": {
                "w": "v",
                "sub": {
                  "older": 4255898
                }
              }
            },
            {
              "multi": {

```



```

↳ (65535),multi(2,[b8688df1/84'/0'/2']
↳ xpub6DNfJehqF1LUwZ1DrFcQN365CDMPLSsrrqpSwaz4eRrtYz9qWdsVH9JqshpQ6yNGdG4VXGKbLgCcxKkXtwTB5B4iCuHcmjomqR8z6NNxNpU51/<0;1>/*,[b8688df1/84'/0'/3']
↳ xpub6DNfJehqF1LUxb8gRSstbR9LcAXWgWd4V678z5FZ7BLftWzwQCT3yZb5eW4U8AVJwbKcZ2ieggjvKv2xggJpMBRk3CE7bz7g4uMV7Qp3TU/<0;1>/*),and_v(v:older(4255898)
↳ ,multi(1,[28645006/84'/0'/0']xpub6DBBzvudcQg2nS4tHskrm7F6kXL6arWxEYoZ1g1GwbaNdWRobcJmxH8KQdjuzTzJtwDveibGd83eGzdGCR6NjSqpU8p2Xx4wWQK7iGBycAm
↳ /<0;1>/*,[28645006/84'/0'/1']xpub6DBBzvudcQg2sPDRWpqfEn5VzPiwrD1zNes4aHmNUog9Jmc2fc2J5fM2E39YzY2iakmpWRpa3rXhzGNmd4GKiJEZxmTcftQpWBGd9ihVks
↳ /<0;1>/*,[28645006/84'/0'/2']xpub6DBBzvudcQg2uk9BrxsuxCwj5YsbfPYkPahQfmTVABfJ4j8TRxUnRd5eGgEXfgPj63xcuTV9uny7pQ8Fbu3XKcN9rNxcNGRaDT9BD1gkBGZ
↳ /<0;1>/*)))#4ld0crxa
/0'/,1/*
bc1q4g7564xxd9hj68hquw5e558cqafhsklerkr0asfzqp6puq74veesrp6qss
note: stdout is watch-only – public keys only, cannot spend

```

5.3 Engrave the card set

Because every key is **distinct**, this is a valid BIP-388 wallet policy, so -- unlike a key-reusing policy -- `bundle` will engrave it. With only public xpubs supplied, the result is watch-only (the `ms1` cards are empty placeholders). The descriptor file it reads:

```

$ cat policy.desc
wsh(or_i(and_v(after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(3,[73c5da0a/84'/0'/0']
↳ xpub6CatWdiZiodmUeTpDLT5or8nmBKNcuyvz7WysVFkKB4RHCD3YuvPEbvqAQY3rAPshwCMLoP2fMFMKHPJ4ZeZXYVUhLv1VMrjPC7PW6V/<0;1>/*,[73c5da0a/84'/0'/1']
↳ xpub6CatWdiZiodmYVtWLTQE5Ag1H9ooS1bmsJUBwQ83FE1Fyk386FWcyicJgEZv3quZSJKASdh5Lo2PpubMGxCfZtRthV6ST2qquL9w3HSzcUn/<0;1>/*,[73c5da0a/84'/0'/2']
↳ xpub6CatWdiZiodmbNgqcQxxjGN165QxTU4PwNNi9WrijYgyf7VxcmuFosRw3focZLgRDbjDjJbqZhPCTfKcaWmL9Bu5w98ybKJtchGweryy6/<0;1>/*)),or_i(and_v(after
↳ (1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(2,[73c5da0a/84'/0'/3']
↳ xpub6CatWdiZiodmHurRokjbycCrxddTDJgTsyEaaQfKjKwbWUi79LAWG5gHjMCQB7BeJc47MkubXuZdf45JZHk1qcr1GZ5EwREUDVDLdVdPkEC/<0;1>/*,[b8688df1/84'/0'/0']
↳ xpub6DNfJehqF1LU59kwaqDu12Ajz9psYVtbGhTykQo1CYdkkqV2yAr4DiWxSTTDujWHzVy1AtV6ENGKwGwLWqa4wXMR4ZmdpRjQBG5EgTV/<0;1>/*,[b8688df1/84'/0'/1']
↳ xpub6DNfJehqF1LUsoh1a4XLG3yuFY1oCRJd6Bkba97xvW65wvY71Nc6LvqfDLWuyCiv4eMDxgFtiTxnazgprYVzQSKR3L8EGRs9asBarqpe/<0;1>/*)),or_i(and_v(older
↳ (65535),multi(2,[b8688df1/84'/0'/2']
↳ xpub6DNfJehqF1LUwZ1DrFcQN365CDMPLSsrrqpSwaz4eRrtYz9qWdsVH9JqshpQ6yNGdG4VXGKbLgCcxKkXtwTB5B4iCuHcmjomqR8z6NNxNpU51/<0;1>/*,[b8688df1/84'/0'/3']
↳ xpub6DNfJehqF1LUxb8gRSstbR9LcAXWgWd4V678z5FZ7BLftWzwQCT3yZb5eW4U8AVJwbKcZ2ieggjvKv2xggJpMBRk3CE7bz7g4uMV7Qp3TU/<0;1>/*),and_v(v:older(4255898)
↳ ,multi(1,[28645006/84'/0'/0']xpub6DBBzvudcQg2nS4tHskrm7F6kXL6arWxEYoZ1g1GwbaNdWRobcJmxH8KQdjuzTzJtwDveibGd83eGzdGCR6NjSqpU8p2Xx4wWQK7iGBycAm
↳ /<0;1>/*,[28645006/84'/0'/1']xpub6DBBzvudcQg2sPDRWpqfEn5VzPiwrD1zNes4aHmNUog9Jmc2fc2J5fM2E39YzY2iakmpWRpa3rXhzGNmd4GKiJEZxmTcftQpWBGd9ihVks
↳ /<0;1>/*,[28645006/84'/0'/2']xpub6DBBzvudcQg2uk9BrxsuxCwj5YsbfPYkPahQfmTVABfJ4j8TRxUnRd5eGgEXfgPj63xcuTV9uny7pQ8Fbu3XKcN9rNxcNGRaDT9BD1gkBGZ
↳ /<0;1>/*)))#4ld0crxa

```

The watch-only card set:

```

$ mnemonic bundle --descriptor-file policy.desc --network mainnet
# ms1 (omitted – descriptor watch-only mode)

# mk1[0] (cosigner 0 xpub + origin)
mk1qptfcrzqq945upmkp8qwastrfchrvz6wqamqkns8wc95upmkp8qwastrfchrvz6wqamqkns8wc95upmkpeutks2qzvzgs7hd8pnnmcv56c4u
mk1qptfcrzpgk08aueetmd998g9tyxuae9vxn38f9gtrp98q8s808l6szjxkt6r83rk2jg0bcqns0f30mtxzd65mwvcur9usv8qhcmu69gql0zrc
mk1qptfcrz74hwqxdp883jehp5tdrfaf0n5zdfhw6425sqe8pgsyxftjt4

mk1qp tfczr qqs94 5upmk p8qwa astfc rhvz6 wqamq kns8w c95up
mkp8d qwastrfchrv z6wqa mqkns 8wc95 upmkp eutks 2qvzgs 7hd8
pnnmc v56c4 u
mk1qp tfczr pkg08 auetm d998g 9tyxu ae9vx n38f9 gtrp9 8qk8s
08l6s zjxkj t6r83 rk2jg 0bcns 0f30m txzd6 5mwvcur9us v8qhc
mu69g ql0zr c
mk1qp tfczr z74hw qxqdp 083je hp5td rfa0n 5zdfh w6425 sqe8p
gsyxftjt4

# mk1[1] (cosigner 1 xpub + origin)
mk1qptfcrzqq945upmkp8qwastrfchrvz6wqamqkns8wc95upmkp8qwastrfchrvz6wqamqkns8wc95upmkpeutks2lcpag7x7w0uuvz59ukc0
mk1qptfcrzpsqzqgqzyqsqqgqzy3zepulhn90du6setz7wcggn0mnt0jarage0ptr0vxf3c9895ghkha0gk58rcwcdclrvxvprnta
mk1qptfcrz74hwqxdp883jehp5tdrfaf0n5zdfhw6425sqe8pgsyxftjt4

mk1qp tfczr qqs94 5upmk p8qwa astfc rhvz6 wqamq kns8w c95up
mkp8d qwastrfchrv z6wqa mqkns 8wc95 upmkp eutks 2lcpa g7x7w
00uvz 59ukc 0
mk1qp tfczr pszqg qzyqs qqgqz ypsqz qqzqy 3zepu lhn90 du6se
tz7wc ggw0m nt0ja rage0 ptr0v xf3c9 895gh kha0g k58rc wcdc
lrxxv vprnt a
mk1qp tfczr zdp8 2uxnp rxcue jaent sxshg r9vud frx5c c7npa
egptf 64nsu d0nnn t334x du0p8 3dn95 g54tz

# mk1[2] (cosigner 2 xpub + origin)
mk1qptfcrzqq945upmkp8qwastrfchrvz6wqamqkns8wc95upmkp8qwastrfchrvz6wqamqkns8wc95upmkpeutks2lcpag07c7z8k5qayd8z5
mk1qptfcrzpsqzqgqzyqsqqgqzy3zepulhn90d76enfqwtug4zdpn8r94nrhgnwyc6l2errqlch68l94cdsr0vxxvngvzwq0fhe86j
mk1qptfcrz74hwqxdp883jehp5tdrfaf0n5zdfhw6425sqe8pgsyxftjt4

mk1qp tfczr qqs94 5upmk p8qwa astfc rhvz6 wqamq kns8w c95up
mkp8d qwastrfchrv z6wqa mqkns 8wc95 upmkp eutks 2lcpa g07c7
z8k5q ayd8z 5
mk1qp tfczr pszqg qzyqs qqgqz ypsqz qqzqy 3zepu lhn90 d76en
fqwtug4zdpn8r9 4nrhg nwycc 6l2er rqlch 68l94 cdsr0 vxvng
vzwq0 fhe86 j
mk1qp tfczr zqupq c34fl c5lnh z7dzk jmwzf htn2t f9tp8 385pw
pa2rh 2p9a6 flzxm hl43h hx3cm gcjp6 25l8l

```

```
# mk1[3] (cosigner 3 xpub + origin)
mk1qptfcqzqs945upmkpd8qwestfcrhvv6wqamqkns8wc95upmkpd8qwestfcrhvv6wqamqkns8wc95upmkpeutks2lcpagqk6f7csue6uzgna
mk1qptfcqzpsqzqgqzyqsqgqzyrszqgqzy3zepulhn90d6awkdsm10jm7xjylhyyp7esaklwkwkasq24gnq3578mne3sjr64gdauj68uhy768d
mk1qptfcqzqqp0dmvslcy68sh0lw93nwwxth488eqjvn67gqjpx7fyf84xj7zjfc29xjtagpghsxz5xlu0rx

mk1qp tfcqz qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkp eutks 2lcpa gqk6f
7csue 6uzgn a
mk1qp tfcqz pszqg qzyqs zqgqz yrszq gqzqy 3zepu lhn90 d6awk
dsm10 jm7xj ylhy p7esa klwk kasq2 4gnq3 578mn e3sjr 64gda
uj68u hy768 d
mk1qp tfcqz zqp0 dmvs1 cy68s h0lw9 3nwwx th488 eqjvn 67gqj
px7fy f84xj 7zjfc 29xjt agpg h szx5x lu0rx

# mk1[4] (cosigner 4 xpub + origin)
mk1qptfc8zqs945upmkpd8qwestfcrhvv6wqamqkns8wc95upmkpd8qwestfcrhvv6wqamqkns8wc95upmkzux3r03qvzgsnwq0e2f7mmwlennx
mk1qptfc8zpkg0w55t7uh3mhpjgm2ctzuevmwdfap2f5z19lkdftfh372tg8y9k3d35qm006q30450k4vmqvakpywv7pkdajnnrd36ymdaqksnv
mk1qptfc8z2m36y9wa86gpqpq5ypppy22y3ww23fxkwgjcskxqsga8sgz6t1

mk1qp tfc8z qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkz ux3r0 3qvzg snwq0
e2f7m mwlnn x
mk1qp tfc8z pkg0w 55t7u h3mhp gjm2c tzuev mwdfp ap2f5 z19lk
dtfh3 72tg8 y9k3d 35qm0 06q30 450k4 vmqva kpywv 7pkda jnnrd
36ymd aqksn v
mk1qp tfc8z zm36y 9wa86 gpqpq 5yppp y22y3 ww23f xkwgjc skxq
sga8s gz6t1

# mk1[5] (cosigner 5 xpub + origin)
mk1qptfcxqzqs945upmkpd8qwestfcrhvv6wqamqkns8wc95upmkpd8qwestfcrhvv6wqamqkns8wc95upmkzux3r03lcpagnlnxhkxfprlea74
mk1qptfcxpszqgqzyqsqgqzyrszqgqzy3zepa6j30mj2615afq22n2ct2h3zwrjpsfeyfqr5zrzs70jdmqtqv7wwt026a3fmdmqr0khu23
mk1qptfcxzzdp6ttmcl57ac6vels0u3jj86dd5pe7m355rttw8r3a2s45tszcmrx9xa5qyec4g3crut8d54

mk1qp tfcxq qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkz ux3r0 3lcpa gnlnx
hkxfp rlea7 4
mk1qp tfcxq pszqg qzyqs zqgqz ypsqz gqzqy 3zepa 6j30m jj261
5afq2 2n2ct 2h3zw rjpse fyfqr 5zrzs 70jdm tqvx7 wwt0z 6a3fm
fdmqr 0khu2 3
mk1qp tfcxq zdp6 ttmc1 y57ac 6vels 0u3jj 86dd5 pe7m3 55rtt
w8r3a 2s45t szcmr x9xa5 qyec4 g3cru t8d54

# mk1[6] (cosigner 6 xpub + origin)
mk1qptfc9zqs945upmkpd8qwestfcrhvv6wqamqkns8wc95upmkpd8qwestfcrhvv6wqamqkns8wc95upmkzux3r03lcpagz84k67v3r27gvvyw
mk1qptfc9zpszqgqzyqsqgqzyrszqgqzy3zepa6j30mjemst4u6j4atq6uhd0m3m5l9lvu9cde6pcswdr73ejss4lts9qugnufy3yg6r2y0f
mk1qptfc9zzvgpdye4j7rexgzc7vsch5dxs9e587e8u22q38ujtrt541m26dq30kfqvm7wyvxtmzy38suff

mk1qp tfc9z qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkz ux3r0 3lcpa gz84k
67v3r 27gvvy w
mk1qp tfc9z pszqg qzyqs zqgqz ypsqz gqzqy 3zepa 6j30m jemst
4u6j4 atq6u hd0m3 m5l9l vu9cd e6pcs wdr7 3ejss 4lts9 qugnu
fy3yg 6r2y0 f
mk1qp tfc9z zvgp dy4j 7rexg zc7vs ch5dx s9e58 7e8u2 2q38u
jtrt5 4lm26 dq30k fqvm 7wyvxtmzy3 8suff

# mk1[7] (cosigner 7 xpub + origin)
mk1qptfcyqzqs945upmkpd8qwestfcrhvv6wqamqkns8wc95upmkpd8qwestfcrhvv6wqamqkns8wc95upmkzux3r03lcpagd0hpxp2e6dx8r48
mk1qptfcyqzpszqgqzyqsqgqzyrszqgqzy3zepa6j30mjstc3wnact9lauqxytd7f9xhqmtmlma4zhsvtx5p6e3s3rggp2ce7mxam45j76f5qd
mk1qptfcyqz3qpzlzuww4vzvcsequdfl7d8u34vrr7t6kyjwlagkep2ywk6g2q5h3z3tv9m06y2yd6kfgs

mk1qp tfcyq qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkz ux3r0 3lcpa gd0hp
xp2e6 dx8r4 8
mk1qp tfcyq pszqg qzyqs zqgqz yrszq gqzqy 3zepa 6j30m jstc3
wnact 9lauq xytd7 f9xhq mtmlm a4zhs vtx5p 6e3s3 rggp2 ce7mx
am45j 76f5q d
mk1qp tfcyq z3qpz lzuww w4vzv ckseq udf17 d8u34 vrr7t 6kyjw
lagke p2ywk 6g2q5 h3z3t vy9m0 6y2yd 6kfgs

# mk1[8] (cosigner 8 xpub + origin)
mk1qptfctzqs945upmkpd8qwestfcrhvv6wqamqkns8wc95upmkpd8qwestfcrhvv6wqamqkns8wc95upmkq5xg5qxqvzgs9n3l96x69pda2ys
mk1qptfctzpkg0dqcwzpsyrk9a808d5f0gennj3x9vjds63rqgzndx78e533rulqyzd58cmvq4nudnky9jhqsh5zpemaskg7pdgxcftukrgv6n5
mk1qptfctz2t4c4w6rtnmhnt889lthvne3ktxs5encnny6spl8sgsf53tn98

mk1qp tftcz qqs94 5upmk pd8qw astfc rhvz6 wqamq kns8w c95up
mkpd8 qwast fcrhv z6wqa mqkns 8wc95 upmkq 5xg5q xqvzg s9n3l
96x69 pda2y s
mk1qp tftcz pkg0d qcwzp syrk9 a808d 5f0ge nnj3x 9vjds 63rqg
zndx7 8e533 rulqy zd58c mvq4n udnky 9jhqs h5zpe maskg 7pdgx
cftuk rgv6n 5
mk1qp tftcz zt4c4 w6rtm nhnt 889lthvne3 kt5e ncny 6spl8
gsqf5 3tn98

# mk1[9] (cosigner 9 xpub + origin)
mk1qptfc2zqs945upmkpd8qwestfcrhvv6wqamqkns8wc95upmkpd8qwestfcrhvv6wqamqkns8wc95upmkq5xg5xqlcpag9z2ktxfdleumyfr
```

```
# === Wallet bundle: descriptor, mainnet ===
# Threshold: 3 of 11
# Cosigners:
# @0: (no ms1; watch-only),mk1:5a703 (73c5da0a @ m/84'/0'/0' )
# @1: (no ms1; watch-only),mk1:5a702 (73c5da0a @ m/84'/0'/1' )
# @2: (no ms1; watch-only),mk1:5a701 (73c5da0a @ m/84'/0'/2' )
# @3: (no ms1; watch-only),mk1:5a700 (73c5da0a @ m/84'/0'/3' )
# @4: (no ms1; watch-only),mk1:5a707 (b6868df1 @ m/84'/0'/0' )
# @5: (no ms1; watch-only),mk1:5a706 (b6868df1 @ m/84'/0'/1' )
```

```
# @6: (no ms1; watch-only),mk1:5a705 (b8688df1 @ m/84'/0'/2')
# @7: (no ms1; watch-only),mk1:5a704 (b8688df1 @ m/84'/0'/3')
# @8: (no ms1; watch-only),mk1:5a70b (28645006 @ m/84'/0'/0')
# @9: (no ms1; watch-only),mk1:5a70a (28645006 @ m/84'/0'/1')
# @10: (no ms1; watch-only),mk1:5a709 (28645006 @ m/84'/0'/2')
# Template: descriptor
# md1: 5a70
# Recovery: any 3 of 11 signing keys + md1 (template card).
note: stdout is watch-only – public keys only, cannot spend
```

5.4 Watch-only export for Nunchuk / Core / Sparrow

There is no dedicated `nunchuk` emitter, but Nunchuk imports miniscript wallets from a **descriptor** or a **BSMS (BIP-129)** record -- both shown above in 5.2 (`--format descriptor` for *Add Wallet -> Import -> descriptor*, and `--format bsms` for the multisig import format, which also Bitcoin Core and Sparrow accept). After import, fund the address; each spend path opens only when its lock matures, and the hashlock tiers additionally require revealing the secret word's preimage `x`.

6. Taproot version of the degrading wallet

with (...) reveals the whole policy on every spend. **Taproot** gives a cooperative **key-path** spend (cheap, private, looks like single-sig) and splits the fallbacks into **script-tree leaves**, so a spend reveals only the leaf it uses. We keep the same four timelock/hash/multisig tiers as fallbacks and add a distinct cooperative internal key k_{int} ([.../84'/0'/4']). Taproot multisig uses **multi a**, not **multi**. **12 distinct keys** in total.

6.1 Why depth-1 (the one-tier-per-leaf limit on master)

The tidiest layout is one tier per leaf (4 leaves), but that is a **depth-2** taptree, and the shipped rust-miniscript pin mis-formats depth->=2 taptrees (the PR-#953 bug). The toolkit refuses such a descriptor up front rather than emit a malformed one. The four-leaf (depth-2) descriptor file:

```
cat taproot-4leaf.desc  
tr([73c5da0a/84/'0'/4']xpub6CatWdizIodmeTdpB8L7S0r8nmbKkNcuYvZ7WyksVFfKB4RHwCD3XyuvPEbvAQOY3rAPshWCmLOPzFmfMKHPJ4ZeZZXYUHLVlVMjrPC7PWGv/<0;/>1/*,[73c5da0a/84/'0'/1'  
    <br><i>→ after(1000000),and_v(v:=sha256(aad4ce49t5727c398023cfbd50d5db39662375521d0f1ac62bdb29b9a08ad),multi_a(3,[73c5da0a/84/'0'/0']  
    <br><i>→ xpub6CatWdizIodmeTdP8L7S0r8nmbKkNcuYvZ7WyksVFfKB4RHwCD3XyuvPEbvAQOY3rAPshWCmLOPzFmfMKHPJ4ZeZZXYUHLVlVMjrPC7PWGv/<0;/>1/*,[73c5da0a/84/'0'/1'  
    <br><i>→ xpub6CatWdizIodmeTgqWLTEQsAgH19oos1bmsJUbw803FE1Fyk386FWcyicjGeZV3quZJKAsDj5Lo2PubMgxCfZtrJbhV6ST2qqUL9w3HSzcUn/<0;/>1/*,[73c5da0a/84/'0'/2'  
    <br><i>→ xpub6CatWdizIodmbNgvcxxjGN16soQTu4PwNNi9WrjiYGyF7VxmccFoxSR3fcz3RJBddJhJbqZhPTfkcaAMLU9BuSW98ybKKZChHgWery6/<0;/>1/*))],and_v(v:=after  
    <br><i>(1893456000),and_v(v:=sha256(aad4ce49t5727c398023cfbd50d5db39662375521d0f1ac62bdb29b9a08ad),multi_a(2,[73c5da0a/84/'0'/3']  
    <br><i>→ xpub6CatWdizIodmdhmRokjbycCxrdTDJgTsEyaAQfKblwbWUl9LaWG5dHiMQB7BeJC47MkubXuZdf45ZHJKtcrIGZE5UEVDUDVLdVPKeC/<0;/>1/*,[b868ddf1/84/'0'/0']  
    <br><i>xpub6DNfJehqFl1Us9kwaQu12Ajpzp9syVzbghTKyK0lCYdkdqV2vayARd1WXSTDTjqWhZVyAt1AV6ENGKGwsbLWqa4wxMZXR4ZmdpRjOBGSegTV/<0;/>1/*,[b868ddf1/84/'0'/1'  
    <br><i>xpub6DNfJehqFl1Usohl1ALGXlgryFYlCRjd6EBka97xvw6SwwY71Nc6LvkgfDLWuyCivJ4eMDxgfLiTxnazgp8rYvZQSKR3LEGRs9asBarqpe/<0;/>1/*)))],{and_v(v:older  
    <br><i>(65535),multi_a(2,[b868ddf1/84/'0'/2']  
    <br><i>→ xpub6DNfJehqFl1UwZldRfCcNQ365CDMPLSRsraqSwaz4ErRtYz9qdwsVH9JqsHp06yNGdvGVXKgBLgcXkKxtWB5B4iCuHcmjomqr8z6MNxnUp51/<0;/>1/*,[b868ddf1/84/'0'/3']  
    <br><i>xpub6DNfJehqFl1UxbgrRS5tbR9LCAXwgvd4V678ZF527FLbtfwzCKt3yZkd5u6UA8VJwbKcz21igebnJdwXogJmcR8K3JEZ7b4gmV7qp3TU/<0;/>1/*)),and_v(v:older(4255898)  
    <br><i>,multi_a(1,[28645006/84/'0']/xpub6DBbzvudcqQgnS24K5m7FGKL6AwLxvQc221gwabJdgBg3mxHBKQJCEJTzTwdeibXGGSGzdG6CR6ljSpq8r2vxXdAWqK7iGbyAcM  
    <br><i>/<0;/>1/*,[28645006/84/'0'/1]xpub6DBbzvudcqQg2SPDRWPqfEn5VzPiwrdl2nes4AHmmNUogJmc2fc2JSfME239YZyiakmpWRpa3rhXzhGnm4GKiIEZxmCftfpQwB9diHVks  
    <br><i>/<0;/>1/*,[28645006/84/'0'/2]xpub6DBbzvudcqQ2uk9BrxsuxCWjsYsbFPYPahQfmTVABfJ4j8TRXUnRds5ecGEkfGPJ63xcuTVun9y7q0BFbu3XKN9rxNXcnGRArD9BD1gkBgz  
    <br><i>/<0;/>1/*))))'
```

Asking the toolkit to export it:

\$ mnemonic export-wallet --descriptor "\$(cat taproot-4leaf.desc)" --format descriptor --network mainnet error: export-wallet script-type derive: taptree branch must have 2 children, but found 1

So we use a `**depth=1**` tree (2 leaves) and pack two tiers per leaf with `'or_i'`:
 Leaf A = tier 1 or tier 2 (the absolute-timelock + secret-word tiers); Leaf B =
 tier 3 or tier 4 (the relative-timelock tiers). (A rust-miniscript release
 > 13.1.0 containing #953 reopens deep trees -- tracked in FOLLOWUP
`taproot-coverage-cycle-on-miniscript-gt-13-1-0`.)

6.2 Build + validate

The hand-written depth-1 `tr(...)` descriptor file:

\$ cat taproot.desc tr([73c5da0a/84'/0'/4']xpub6CatWdiZiodmeXswr13Gd5aNtNqr2UHCBEsCoL3eEFVaM7n8kY5kS4daaP83gWC

Validate and canonicalise it. The full canonical descriptor, every xpub in full:

```
$ mnemonic export-wallet --descriptor "$(cat taproot.desc)" --format descriptor --network mainnet
tr[73c5da0a/84/'0/'1']xpub6CatWdIzIodmeXswr13Gd5AtNtqr2CH0EBCEs3Kf0aEEFvAm7n8kY5k54d6aP83jqWqncmzL3Wzt79mEiIX6XZs6XGQCnGcQ4HcJfVtN9tUXe/;<0>;1/*,[{or_i(
  and_v(v/after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd05d5db3662375521d0f1ac62dbd829b9a08ad),multi_a_3,[73c5da0a/84/'0/'0']
  )xpub6CatWdIzIodmeUdPb8LT5or8nmbKkcuyvz7WkysVFkKB4RHwCD3xyuvPEbvqAQY3rAPshwCmLP02PfMFMKHPJ4ZeZXYVUHLv1VMrjPC7PWW6v/;<0>;1/*,[73c5da0a/84/'0/'1']
  )xpub6CatWdIzIodmYvqLwTEQsAg1H9oos1bmsJbW8d83FE1Fyk386FWciyCjGzV3quZ3JKB5dJ5Lo2PubbMGxCFzTzRhtV6ST2qqUL9w3HSzyc/;<0>;1/*,[73c5da0a/84/'0/'2']
  )xpub6CatWdIzIodmGgXocxjGN650xtU4PAmN19wJrYgYV7VxmCufxosRw3foc2ZKJd7Jq3bZnPCTfkcaWmL9Bu5w98ybqChWgEgV/;<0>;1/*,[{and_v(v:after(
    1893456000),and_v(v:sha256(a84dce40975727c398023cfbd05d5db3662375521d0f1ac62dbd829b9a08ad),multi_a_2,[73c5da0a/84/'0/'3']
    )xpub6CatWdIzIodmHuvRokjbyCrxdtdJgT5tYEAQ4KjkbWuq179AWG5gHjMCQ87BeJz47MkubUzXdf45JZHk1qcr1GZ5eWUdEVDJLdVPKec/;<0>;1/*,[b688df1/84/'0/'0']
    )xpub6DNfJehqf1LU59kwaqu012Ajz3p9ytsYvBghTqKfQ01CydkqV2vayRg4d1WXSTTDUj7q4V1atV6ENGKwgwbLUx4qWmZr4ZmdpRjQBG5EgV/;<0>;1/*,[b688df1/84/'0/'1']
    )xpub6DNfJehqf1LU5oeh14x3GyFuY1LoCRJd6Bkba97xv6W5wvY71Nc6LvqkfDLWuyCivJ4eMDxgf1Xnzagp8rYVzQSKR3L8GRs9asBarqpe/;<0>;1/*,[{or_i(and_v(v:older
      (65535),multi_a_2,[b688df1/84/'0/'2']
      )xpub6DNfJehqf1LUwZrFrcQn365CDMPsLsrpSvaz4eRrtYz9qWdsVH9JqsHp06yNGdG4VXGkbLgcXkKxTwtB5B4iCuHcmj0mqR8z6NNXnUp51/;<0>;1/*,[b688df1/84/'0/'3']
      )xpub6DNfJehqf1LUx8R5stbR9LcAXgw6d4V6785ZF2bLftwzCQ3t3y25ew4U8VJwbKcZ21gwgJjyKvZ2qjPMRRkKJCEZt7g4uW7q3TU/;<0>;1/*,[{and_v(v:older(4255898)
        ,multi_a_1,[28645006/84/'0/'0']xpub6DBbzvudCqg2nS45Hskmr7FGKXL6arG4vG0221gEabNwR0bJ3mXkBGK03U72tVdweibGd83eGd6CR6RjSqP8u2vx4dAwQK7iG6yAm
        )/;<0>;1/*,[28645006/84/'0/'1']xpub6DBbzvudCqg2sPDRWpqfEn5YvPiwrd1zNes4aHmnnUogJmc2fc23SFmE239Yz2iakmpWRpa3rXhZGnm4GKiJEZxmCftfQpWBG9dhVks
        )/;<0>;1/*,[28645006/84/'0/'2']xpub6DBbzvudCqg2uk9BrxsuxCwjsYsbfPYkPaHqfmTVABf4J748TRxUnRd5eGgXfpgJ633xctUv9ny7qP8Bbu3XKCN9rNxcNGRABT9BD1gkBGZ
        )/;<0>;1/*,[{snerswx7
```

note: stdout is watch-only – public keys only, cannot spend

`Kind` (`[73c5da0a/84'/0'/4']`) is the key-path; the two `or_i(...)` blocks are the two script leaves; `after(...)` are the absolute (height/time) locks and `older(...)` the relative (blocks/time) locks -- the same four kinds as section 5.

6.3 Engrave + first address

Every key is distinct, so it engraves (watch-only). Take the md1 chunks and restore to read the first address (this round-trip also proves the **real internal key** at the trunk reconstructs -- the v0.55.3 non-NUMS feature). The descriptor file (shown again):

```
cat taproot.desc
tr([73c5da0a/84/0/0']xpub6CatWdiZiomdeXswr13Gd5s1Nqr2UHCBEsCoL3eEFVaM7n8Yk5K54daaP83gwQncmzL3Wzt79mEiLiX6XZ36X0mGcQcNeQ4HcjfVtN9tUXE/<0;1>*/,{or_i(
  _and_v/{after(1000000)},_and_v/{v:sha256(a84dec4a9975727c398023cfbd50d5db39662375521d0f1ac62dbd829b9a08ad),multi_a_3,[73c5da0a/84/0/0']
  xpub6CatWdiZiomdeUpDb8LT5or8nmbKkcuyvz7WmYksVFKB4RHwCD3xyuvPEbvAQYQ3rAPshwCmLoP2fMFMKHPJ4ZeZXYUHLv1VMrjPC7PW6V/<0;1>*,[73c5da0a/84/0/0']
  xpub6CatWdiZiomdYvqLwTEQsAg1H9oos1bmsJUBw8Q3FE1Fyk386FWcYicJgZv3qzL3KqJ5Adh5D2PbubMGxCfZtR7hV6Sty2kqL9w3HSzcUn/<0;1>*,[73c5da0a/84/0/0']
  xpub6CatWdiZiomdYvqCxxjG1n650xtU4Pw6Nm19wriYgY7VfxcmufxosRg3focZGRBj4J7JbqZhPCfkcaWmL9Bsu5872KQJLChgWeyV/<0;1>*/)},_and_v/{after
  (1893456000)},_and_v/{v:sha256(a84dec4a9975727c398023cfbd50d5db39662375521d0f1ac62dbd829b9a08ad),multi_a_2,[73c5da0a/84/0/0']
  xpub6CatWdiZiomdHurvKokjbycCrxdDTJgTsYEAaQfKkIkbWbU179ALMG5gHjMC0B7BeJz47MkubX4d5JZHK1qcr1GZ5wEuTiDVLdVPkEc/<0;1>*,[b8688df1/84/0/0']
  xpub6DNfJehqf1LU9sKwaqjD21ajzp9sYVvbgHtYKfQ01CydkdqV2ayARd1WXSTDuJ7WbZJ1atV6ENGKwgbWLUq4wXMR24ZmdpRJQBG5EGV/<0;1>*,[b8688df1/84/0/0']
  xpub6DNfJehqf1LUsoh1aXLG3fyuF1LoCRd6Bkba97xvW6SwwY71Nc6LvqkfDLWuyCivJ4eMDXgFtiXnazgp8rYVZQSKR18EGRS9asBarqpe/<0;1>*/)},or_i(_and_v/{older
  (65535)},multi_a_2,[b8688df1/84/0/0']
  xpub6DNfJehqf1LUwZ1DrFcQN365CDMLPSrwapSwaz4eRrtYz9qWsdVH9JqsHp06YNgDG4VXGKBGcxKkXtWtB5B4iCuHcmj0mqR8z6NNXnUp5U/<0;1>*,[b8688df1/84/0/0']
  xpub6DNfJehqf1LUx8hR5stbR9LCaXwgw4V678z5FZ7BLftwzCQ3tYzB5ew4U8AVJwbKcZ21gwgjYkVzvggJPMRRKk3Jez7bZ4gmV7Qp3TU/<0;1>*/)},_and_v/{older(425589)
  },multi_a_1,[28645006/84/0/0']xpub6DBbzbvudCqg2n54THskmr7FGKXL6aRvG5o21gJmWdMxR0bJ3meXh8K0CUEZ72tDvweiB3Gz6zGCR6Rj5apU8p24awXWQ07iGbgYm
  /<0;1>*,[28645006/84/0/0']xpub6DBbzbvudCqg2sPDRWpqpEn5VzPzr1dZnes4aHmMNUog9Jmc2fc23SfM2E39Yz2iakmpWRpa3rXhZGmnd4GK1iEZxmTcftQpWBg9ihVks
  /<0;1>*,[28645006/84/0/0']xpub6DBbzbvudCqg2uk9BrsxuxCwjsYsbFPYkPahQfmTVABfJ4j8TRXUnRds5eGgEXfgPJ63xcuTVun9y7pQBFb3u3KKCn9rNxcNGRaDT9BD1gkBGZ
  /<0;1>*/)}))`
```

```
$ mnemonic bundle --descriptor-file taproot.desc --network mainnet --json | jq -r ".md1[]" > taproot.md1 note: stdout is watch-only — public keys only, cannot spend
```

\$ cat taproot.md1 md1f40gpvq9zem6jzwrh4yymp6jzxw75sj3m6jzt802ggrh4yyvaafp9rh4yykw75ss802ggemcn9wrf4m7m5yt9
md1f40gpvq2jz2sqrqgy2efntvqq7sjqfntk5ymnjqjatj0sucqg70h4gdtkemje3rw4fp6rc6cnnxqh4hnx7rv md1f40gpvqscmmmq5mngy26g
md1f40gpvq6gws7xkx9k7c9xu6pzjkjqg9vfeintsqqpllyqshsnxhqggrcf5gqzn2cvasqu79cwukltfwjcnckn md1f40gpvprg9pw0za5z3883w6
md1f40gpvp65qx52ry2qrt9pj9qpskufqq44hccjz40ltn8tdswlxxk6gs50wsfd6vu4jev3eggy8f0nxdxa6x md1f40gpvp56glt4vs3t2qswftl
md1f40gpvp622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8q7nzlkkvymsvlrcugrdkju72 md1f40gpvz9fkca3cxteqm6l
md1f40gpvz0ptr0vxf3c9895ghkha0gk58rcw6qzw4cdxzd3en9mnxhqdpwsx2ec6jxdf33axqmwg8tzlv563mv md1f40gpvznmjszkn4t8p
md1f40gpvzc7csryzx48lzn7wute526tvfexawdfdy4vy7y7s9c84gwagyhfh8ugmwft46ekqd54ka5c8c48mx md1f40gpvr0a7t0c6gn7uss
md1f40gpvrvmn3jaafe7gyny7hjyqsfhjzfaf5hs5jwzh3mhpjgm2ctzuevmwdfap2f5zl9lsqqkg20rz8nj9q md1f40gpvrkdtfh372tg8y9k3d3d
md1f40gpvr6zg55fzuu4z339d06w5s99f4v94tc38peqc5vjysp6pp3g08exa4sxr0889h3ddqqnqhdl2vm2eucf md1f40gpvpyp6ttmcly57ac6v
md1f40gpvyg6uhd0m3m5l9lvu9cde6pcswdrn73ejss4lts9qcszq6fnt9u8jvs93uep30g6dqq0k3hj4xq9v7xn md1f40gpvyning0aj0c55pz0e
md1f40gpvyckdgr4nrpxssz4jyqytut3ee64sfnz6ryr348le5ljx4sv0e02cjfml4zmy9g36c6spmukzwf5w5f md1f40gpvr9fpgzjjzpmz7nhnk6
md1f40gpvr9tzzetsgt6pqua7cty0t4c4w6rtnjmnht889ltvhne3ktx5etuhl2rmxpaqj6ey0lg6d4mp5l9hjrne md1f40gpvr95xj46th52vv3c3kju
md1f40gpvr9evh2tqvnhxzcck5kfhaci7d2lxnax2yvwfue5grx4rgemhg0nk5eg0ze8tj0qfmr3syntuit2igx5pl md1f40gpvx92k0c250h62ypsar

```
$ mnemonic restore --network mainnet $(sed "s/~/--md1 /" taproot.md1) miniscript policy restore (12
cosigners) CONFIRM: verify each cosigner fingerprint against your records before importing. descriptor:
tr([73c5da0a/84'/0'/4']xpub661MyMwAqRbcEyUKSqsBgazlLob8pCa1rM1SJ8CEzGCYyP9LisxZ2m1goDqj137XvHdY2nNkctqj1
first recv: bc1p9stcwz5597fmkxae9343k8edzkcvcdf9qp65r6p447pg0et82yqst3d2c cosigner @0: 73c5da0a [84'/0'/4']
from md1 (not independently verified) cosigner @1: 73c5da0a [84'/0'/0'] from md1 (not independently verified)
cosigner @2: 73c5da0a [84'/0'/1'] from md1 (not independently verified) cosigner @3: 73c5da0a [84'/0'/2'] from
md1 (not independently verified) cosigner @4: 73c5da0a [84'/0'/3'] from md1 (not independently verified) cosigner
@5: b8688df1 [84'/0'/0'] from md1 (not independently verified) cosigner @6: b8688df1 [84'/0'/1'] from md1 (not
independently verified) cosigner @7: b8688df1 [84'/0'/2'] from md1 (not independently verified) cosigner @8: b8688df1
[84'/0'/3'] from md1 (not independently verified) cosigner @9: 28645006 [84'/0'/0'] from md1 (not independently
verified) cosigner @10: 28645006 [84'/0'/1'] from md1 (not independently verified) cosigner @11: 28645006 [84'/0'/2']
from md1 (not independently verified) UNVERIFIED: no --from/--cosigner cross-check supplied; verify each cosigner
fingerprint above against your records before importing note: stdout is watch-only — public keys only, cannot spend
```

(Restore re-serialises each key as a depth-0 `xpub661My...` -- a different descriptor string, identical addresses; that is how the mdl card stores keys.)

6.4 Export for wallets (Nunchuk / Core / Sparrow)

```
`descriptor` and `bitcoin-core` both work for taproot. The descriptor file
(shown again):
```

```
$ cat taproot.desc tr([73c5da0a/84'/0'/4']xpub6CatWdiZiodmeXswr13Gd5aNtNqr2UHCBEsCoL3eEFVaM7n8kY5kS4daaP83gW
```

Bitcoin Core importdescriptors payload:

```
$ mnemonic export-wallet --descriptor "$(cat taproot.desc)" --format bitcoin-core --network mainnet
[
  {
    "active": true,
    "desc": "tr{[73c5da0a/84'/0'/4']xpub6CatWdiZiodmeXswr13Gd5aNtNqr2UHCBEsCoL3eEFVaM7n8kY5kS4daaP83gWQncmzL3Wzt79mEiLiX6XZs6XQmGcQNeQ4HcjfvTn9TuXe/0'/*,{
      ↳ or_i(and_v(v:after(1000000)),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),multi_a(3,[73c5da0a/84'/0'/0'])
      ↳ xpub6CatWdiZiodmUeTdP8LT5or8nmbKNcuyvz7WYksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshwCMLoP2fMFMKHPJ4ZeZXYVUHLv1VMrjPC7PW6V/0'/*,[73c5da0a/84'/0'/1']
      ↳ xpub6CatWdiZiodmYVtWLTeqsAg1H9oo5lBmsJUBwQ83FE1Fyk386FWcyicJgEzV3quZS3KA5dh5Lo2PbubMGxCfZtRthV6ST2qquL9w3HSzcUn/0'/*,[73c5da0a/84'/0'/2']
      ↳ xpub6CatWdiZiodmbNgqcQxxjGN165QxTU4PwNNi9WrijYgYf7VxcmuFxsRw3focZLgRDbjDjJbqZhPCTfkcaWmL9BuSw98ybKKJtcHgWerry6/0'/*))},and_v(v:after(1893456000)
      ↳ ,and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),multi_a(2,[73c5da0a/84'/0'/3'])
      ↳ xpub6CatWdiZiodmHurRokjbycCrxddTDJgTsyEAAQfKjKwbWUi79LAWG5gHjMCQB7BeJc47MkubXuZdf45JZHk1qcr1GZ5EwREUDVDLVdPKEC/0'/*,[b8688df1/84'/0'/0']
      ↳ xpub6DNfJehqF1LU9kwaqDu12Ajz9psYVtbGhTykQo1CYdkkqV2vAyR4DiWXSTTDujWHZVy1AtV6ENGKwgbLWqa4wXMZr4ZmdpRjQBG5EgTV/0'/*,[b8688df1/84'/0'/1']
      ↳ xpub6DNfJehqF1LUsoh1a4XLG3yuFY1oCRJd6Bkba97xvW6SwY71Nc6LvkqFDLWuyCivJ4eMDxgFtiTxnazgp8rYvZQSKR3L8EGRs9asBarqe/0'/*))},or_i(and_v(v:older
      ↳ (65535)),multi_a(2,[b8688df1/84'/0'/2'])
      ↳ xpub6DNfJehqF1LUwZ1DrFcQN365CDMPLSsrrpSwaz4eRrtYz9qWdsVH9JqshpQ6yNgdG4VXGkbLgxcKkXtwTB5B4iCuHcmjomqR8z6NNxNpU51/0'/*,[b8688df1/84'/0'/3']
      ↳ xpub6DNfJehqF1LUxb8gRSstbR9LcAxWgwD4V678z5FZ7BLftWzWqCt3yZb5eW4U8AVJwbKcZ2ieggjvKv2xggJpMBRk3CE7bz7g4uMV7Qp3TU/0'/*)),and_v(v:older(4255898)),
      ↳ multi_a(1,[28645006/84'/0'/0'])xpub6DBBzvudcQg2nS4tHSkrm7FGkX6arWxEYoZ1g1GWbaNdwRobcJmxH8KQdjuzTJ2twDveibGd83eGzdGCR6NjSqpU8p2Xx4wWQK7iGBycAm
      ↳ /0'/*,[28645006/84'/0'/1']xpub6DBBzvudcQg2sPDRWpqfEn5VzPiwrd1zNes4aHmNUog9Jmc2f2c2J5fM2E39Yzy2iakmpWRpa3rXhzGNmd4GKiJEZxmTCftQpwBGd9ihvKs
      ↳ /0'/*,[28645006/84'/0'/2']xpub6DBBzvudcQg2uk9BrxsuxCWjsYsbfPYkPahQfTVABFJ4j8TRxUnRd5eGgEXfgPJ63xcuTV9uny7pQBFbu3XKCn9rNxcNGRaDT9BD1gkBGZ/0'/*))
      ↳ })#q3s6u2uk",
    "internal": false,
    "range": [
      0,
      999
    ],
    "timestamp": 0
  },
  {
    "active": true,
    "desc": "tr{[73c5da0a/84'/0'/4']xpub6CatWdiZiodmeXswr13Gd5aNtNqr2UHCBEsCoL3eEFVaM7n8kY5kS4daaP83gWQncmzL3Wzt79mEiLiX6XZs6XQmGcQNeQ4HcjfvTn9TuXe/1'/*,{
      ↳ or_i(and_v(v:after(1000000)),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),multi_a(3,[73c5da0a/84'/0'/0'])
      ↳ xpub6CatWdiZiodmUeTdP8LT5or8nmbKNcuyvz7WYksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshwCMLoP2fMFMKHPJ4ZeZXYVUHLv1VMrjPC7PW6V/1'/*,[73c5da0a/84'/0'/1']
      ↳ xpub6CatWdiZiodmYVtWLTeqsAg1H9oo5lBmsJUBwQ83FE1Fyk386FWcyicJgEzV3quZS3KA5dh5Lo2PbubMGxCfZtRthV6ST2qquL9w3HSzcUn/1'/*,[73c5da0a/84'/0'/2']
      ↳ xpub6CatWdiZiodmbNgqcQxxjGN165QxTU4PwNNi9WrijYgYf7VxcmuFxsRw3focZLgRDbjDjJbqZhPCTfkcaWmL9BuSw98ybKKJtcHgWerry6/1'/*))},and_v(v:after(1893456000)
      ↳ ,and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),multi_a(2,[73c5da0a/84'/0'/3'])
      ↳ xpub6CatWdiZiodmHurRokjbycCrxddTDJgTsyEAAQfKjKwbWUi79LAWG5gHjMCQB7BeJc47MkubXuZdf45JZHk1qcr1GZ5EwREUDVDLVdPKEC/1'/*,[b8688df1/84'/0'/0']
      ↳ xpub6DNfJehqF1LU9kwaqDu12Ajz9psYVtbGhTykQo1CYdkkqV2vAyR4DiWXSTTDujWHZVy1AtV6ENGKwgbLWqa4wXMZr4ZmdpRjQBG5EgTV/1'/*,[b8688df1/84'/0'/1']
      ↳ xpub6DNfJehqF1LUsoh1a4XLG3yuFY1oCRJd6Bkba97xvW6SwY71Nc6LvkqFDLWuyCivJ4eMDxgFtiTxnazgp8rYvZQSKR3L8EGRs9asBarqe/1'/*))},or_i(and_v(v:older
      ↳ (65535)),multi_a(2,[b8688df1/84'/0'/2'])
      ↳ xpub6DNfJehqF1LUwZ1DrFcQN365CDMPLSsrrpSwaz4eRrtYz9qWdsVH9JqshpQ6yNgdG4VXGkbLgxcKkXtwTB5B4iCuHcmjomqR8z6NNxNpU51/1'/*,[b8688df1/84'/0'/3']
      ↳ xpub6DNfJehqF1LUxb8gRSstbR9LcAxWgwD4V678z5FZ7BLftWzWqCt3yZb5eW4U8AVJwbKcZ2ieggjvKv2xggJpMBRk3CE7bz7g4uMV7Qp3TU/1'/*)),and_v(v:older(4255898)),
      ↳ multi_a(1,[28645006/84'/0'/0'])xpub6DBBzvudcQg2nS4tHSkrm7FGkX6arWxEYoZ1g1GWbaNdwRobcJmxH8KQdjuzTJ2twDveibGd83eGzdGCR6NjSqpU8p2Xx4wWQK7iGBycAm
      ↳ /1'/*,[28645006/84'/0'/1']xpub6DBBzvudcQg2sPDRWpqfEn5VzPiwrd1zNes4aHmNUog9Jmc2f2c2J5fM2E39Yzy2iakmpWRpa3rXhzGNmd4GKiJEZxmTCftQpwBGd9ihvKs
      ↳ /1'/*,[28645006/84'/0'/2']xpub6DBBzvudcQg2uk9BrxsuxCWjsYsbfPYkPahQfTVABFJ4j8TRxUnRd5eGgEXfgPJ63xcuTV9uny7pQBFbu3XKCn9rNxcNGRaDT9BD1gkBGZ/1'/*))
      ↳ })#zdvdstsr",
    "internal": true,
    "range": [
      0,
      999
    ],
    "timestamp": 0
  }
]
note: stdout is watch-only – public keys only, cannot spend
```

But **BSMS / BIP-129** is not available for taproot (BIP-129 predates BIP-386) -- the toolkit says so and points you elsewhere:

```
$ mnemonic export-wallet --descriptor "$(cat taproot.desc)" --format bsms --network mainnet
error: --format bsms does not support taproot (P2trMulti); BIP-129 $! prerequisites do not yet include BIP-386. Real emit support is tracked at FOLLOWUP
↳ 'bsms-taproot-emit' and depends on a BIP-129 spec update. Use --format bitcoin-core (Core-importable) or --format sparrow (Sparrow JSON, taproot
↳ -capable) for taproot watch-only setup.
```

6.5 Adding a condition: which depth? (a spending-cost comparison)

Suppose you want a fifth spend path: a **new key Knew plus the preimage of a RIPEMD-160 hashlock** (secret word "please"). Should it be **folded** into an existing leaf (tree stays depth-1) or given its **own leaf** (forcing depth-2)? Decide on spending cost, not aesthetics. A taproot script-path witness costs: (satisfaction) + the **revealed leaf script** + a **control block** of $33 + 32 \cdot \text{depth}$ bytes. Folding many conditions into one leaf bloats *every* spend through it (you reveal the unused branches too); a deeper leaf adds one 32-byte hash per level. A witness byte weighs 1 WU = 0.25 vB, so **one extra depth level = +8 vB**.

The new hashlock (same two-step scheme): preimage $x = \text{sha256}(\text{"please"})$, descriptor hash $H_p = \text{ripemd160}(X)$:

```
$ python3 -c "import hashlib; X=hashlib.sha256(b'please').digest(); print('preimage X =', X.hex()); print('hash Hp =', hashlib.new('ripemd160', X).
↳ hexdigest())"
```

```
preimage X = 56cccc4dcfc96534b06fc0c08a301be24f13b491484d5d984953cc0dba9bbb89a
hash Hp     = 06d05e2f02fb90ddf98d8cd95d806ba12b27aff4
```

mnemonic compare-cost reports per-condition witness vbytes (key-agnostic -- abstract labels A,B,... are auto-dummy-keyed).
Folded -- the new tier joins Leaf B, so spending it reveals all of tiers 3+4+5:

```
$ mnemonic compare-cost --miniscript "or_i(and_v(v:older(65535),multi(2,A,B)),or_i(and_v(v:older(4255898),multi(1,C,D,E)),and_v(v:pk(F),ripemd160(06
  ↳ d05e2f02fb90ddf98d8cd95d806ba12b27aff4))))"
Input: or_i(and_v(v:older(65535),multi(2,A,B)),or_i(and_v(v:older(4255898),multi(1,C,D,E)),and_v(v:pk(F),ripemd160(06
  ↳ d05e2f02fb90ddf98d8cd95d806ba12b27aff4))))
Wrapper comparison: wsh(M) vs tr(NUMS, {M})
Feerate: 1.0 sat/vB

Condition          | wsh vB | tr vB | Δ vB | wsh sats | tr sats | Δ sats
-----+-----+-----+-----+-----+-----+-----
A + B + older(blocks) | 79 | 147 | +68 | 79 | 147 | +68
C + older(512s)       | 61 | 132 | +71 | 61 | 132 | +71
D + older(512s)       | 61 | 132 | +71 | 61 | 132 | +71
E + older(512s)       | 61 | 132 | +71 | 61 | 132 | +71
F + preimage(h0)      | 69 | 139 | +70 | 69 | 139 | +70
```

note: per-condition vbytes are rounded individually; absolute numbers may differ by ±1 from real-tx accounting, Δ values are correct
note: input contains hash-preimage fragments; preimage-known rows are enumerated assuming the user can supply each preimage (cost only – no preimage
↳ knowledge is implied)

Dedicated -- the new tier is its own leaf, revealing only itself:

```
$ mnemonic compare-cost --miniscript "and_v(v:pk(F),ripemd160(06d05e2f02fb90ddf98d8cd95d806ba12b27aff4))"
Input: and_v(v:pk(F),ripemd160(06d05e2f02fb90ddf98d8cd95d806ba12b27aff4))
Wrapper comparison: wsh(M) vs tr(NUMS, {M})
Feerate: 1.0 sat/vB
```

```
Condition          | wsh vB | tr vB | Δ vB | wsh sats | tr sats | Δ sats
-----+-----+-----+-----+-----+-----+-----
F + preimage(h0)   | 68 | 90 | +22 | 68 | 90 | +22
```

note: per-condition vbytes are rounded individually; absolute numbers may differ by ±1 from real-tx accounting, Δ values are correct
note: input contains hash-preimage fragments; preimage-known rows are enumerated assuming the user can supply each preimage (cost only – no preimage
↳ knowledge is implied)

compare-cost models each input as a single leaf at the tree root (depth-0, 33-byte control block); add the real Merkle depth (+8 vB per level):

Placement	revealed script	tr vB (depth-0)	real depth	+ control block	real tr
Folded into	whole	139	1 (still 2	+8	~147 vB
Leaf B	or_i(t3,t4,t5)		leaves)		
Dedicated leaf	just t5	90	2 (now 3	+16	~106 vB
			leaves)		

A dedicated leaf is ~41 vB (~28%) cheaper to spend the new condition: not revealing tiers 3+4 (~49 vB) beats the +8 vB for the extra depth level. It is better still in practice because folding also makes *t3* and *t4* spends reveal *t5*'s script, and a 3-leaf tree can place the **most-used** condition in the shallow slot and bury cold paths deeper.

Rule of thumb: in taproot, one-condition-per-leaf almost always wins -- each sibling script you avoid revealing is worth far more than the 8 vB/level of depth -- and you order leaves hot-shallow / cold-deep.

The catch: cost says depth-2, but master cannot build depth->=2 yet (the PR-#953 taptree-Display bug, 6.1), so today the only buildable option is the folded depth-1 leaf at the ~28% premium. That premium is the concrete, quantified motive for landing the upstream fix.

Appendix A -- the public test seeds used

Slot	BIP-39 test phrase	Fingerprint (BIP-87)
@0	abandon abandon abandon abandon abandon abandon ↳ abandon abandon abandon abandon abandon about	73c5da0a
@1	legal winner thank year wave sausage worth ↳ useful legal winner thank yellow	b8688df1
@2	letter advice cage absurd amount doctor acoustic ↳ avoid letter advice cage above	28645006

These are world-known BIP-39 vectors with no funds. Sections 5-6 derive their keys from these same three seeds at distinct `m/84'/0'/N'` accounts. Generated with `mnemonic v0.55.3` on Linux. See the in-repo manual (`docs/manual/`) for the authoritative per-flag reference.

Appendix B -- EXPERIMENTAL: depth->=2 taproot reconstruction (NOT FOR REAL FUNDS)

EXPERIMENTAL -- DO NOT USE FOR REAL FUNDS. Everything in this appendix uses `mnemonic` $\hookrightarrow$ `depth2`, a proof-of-concept binary built from the never-merged branch `experimental/taproot-depth-ge2`. It pins an **unreleased** rust-miniscript commit (the PR-#953 merge, in no crates.io release) to lift the `depth->=2` taptree cap. It is **not** part of any shipped release, is not on the install path, and must never be used to secure real funds. The shipped `mnemonic` v0.55.3 documented everywhere else in this guide deliberately **refuses** `depth->=2` (sections 6.1 and below); that refusal is the supported behavior until a rust-miniscript release > 13.1.0 ships #953.

Master refuses to *restore* a `depth->=2` taptree because the shipped miniscript pin mis-formats nested taptrees. The POC bumps that pin and lifts the cap. Build it under a **distinct name** so it never overwrites your real `mnemonic` (both report the same version string 0.55.3, so the command name is the only thing that tells them apart):

```
$ git clone https://github.com/bg002h/mnemonic-toolkit && cd mnemonic-toolkit
$ git checkout experimental/taproot-depth-ge2
$ cargo build --release --manifest-path crates/mnemonic-toolkit/Cargo.toml
$ cp target/release/mnemonic ~/.cargo/bin/mnemonic-depth2 # DISTINCT name
```

Same version string, different binaries -- tell them apart by command name:

```
$ mnemonic --version
mnemonic 0.55.3
```

```
$ mnemonic-depth2 --version
mnemonic 0.55.3
```

Recall the `depth-2` four-leaf descriptor from section 6.1 (one tier per leaf):

```
$ cat taproot-4leaf.desc
tr([73c5da0a/84'/0'/4']xpub6CatWdiZiodmeXswr13Gd5aNTNqr2UHCBEsCoL3eEFVaM7n8kY5kS4daaP83gWQncmzL3Wzt79mEiLiX6XZs6XQmGcQNeQ4HcjfVTN9TuXE/<0;1>/*,{and_v(v:
  after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(3,[73c5da0a/84'/0'/0']
  xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuYvz7WYksVFkKB4RHwCd3XyuvPEbvqAQY3rAPshWcMLoP2fMFMKHPJ4ZeZXYVUhLv1VMrjPC7PW6V/<0;1>/*,[73c5da0a/84'/0'/1']
  xpub6CatWdiZiodmYVtWtLEqSagIh9ooS1bmsJUBwQ83FE1Fyk386FWcyicJgEZv3quZ5JKA5dh5Lo2PbubMGxPfZtRthV6ST2qquL9w3H5zcUn/<0;1>/*,[73c5da0a/84'/0'/2']
  xpub6CatWdiZiodmbNGqcQxxjGN165QxTU4PwNNi9WrijYgYf7VxcmuFxsRw3focZLgRDbjDjJbqZhPCTfkcaWmL9BuSw98ybKKJtcHgweryy6/<0;1>/*)),and_v(v:after
  (1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(2,[73c5da0a/84'/0'/3']
  xpub6CatWdiZiodmHurrRokjbyccrXddTDJgTsyEaAqfKjkbwUi79LAWG5gHjMCQ87BeJc47MkubXuZdf45JZHk1qcr1GZ5EwREUDVDLvdPkEC/<0;1>/*,[b8688df1/84'/0'/0']
  xpub6DNfJehqF1LU9kwaqDu12Ajpz9psYVtbGhTykQo1CYdkkqV2vAyR4DiWXSTTDujWHZVy1AtV6ENGKwGwBLWqa4wXMR4ZmdpRjQBG5EgTV/<0;1>/*,[b8688df1/84'/0'/1']
  xpub6DNfJehqF1LUsohl1aXLG3yuFY1oCRJd6Bkba97xvW6SwwY71Nc6LvkkfDLWuyC1vJ4eMDXgftiTxnazgp8rYvZQSKR3L8EGRS9asBarqpe/<0;1>/*)),{and_v(v:older
  (65535),multi_a(2,[b8688df1/84'/0'/2']
  xpub6DNfJehqF1LUwZ1DrFcQN365CDMPLSsrqpSwaz4eRrtYz9qWdsVH9JqshP06yNGdG4VXGKbLgCkKxTwTB584iCuHcmjomqR8z6NNxNpU51/<0;1>/*,[b8688df1/84'/0'/3']
  xpub6DNfJehqF1LUxb8gRSstbR9LcAXWgwD4V678z5FZ7BLftWzwCt3yZb5eW4U8AVJwbKcZ2ieggjvKv2xggJpMBRk3CE7bz7g4uMV70p3TU/<0;1>/*)),and_v(v:older(4255898)
  ,multi_a(1,[28645006/84'/0'/0']xpub6DBBzvudcQg2nS4tH5Krm7F6KXL6arWxEYoZ1g1GWbaNdWRobcJmxH8KQdjuzTZJtwDveibGd83eGzdgCR6NjSqpu8P2Xx4wWQK7iGBycAm
  /<0;1>/*,[28645006/84'/0'/1']xpub6DBBzvudcQg2sPDRWpqfEn5VzPiwrd1zNes4aHmMNUog9Jmc2fc2J5FM2E39Yzy2iakmpWRpa3rXhzGNmd4GKiJEZxmTCftQpwBgD9ihVks
  /<0;1>/*,[28645006/84'/0'/2']xpub6DBBzvudcQg2uk9BrxsuxCwjsYsbfPykPahQfmTVABfJ4j8TRxUnRd5eGgEXfgPj63xcuTV9uny7pQBFbu3XKCNrNxcNGRaDT9BD1gkBGZ
  /<0;1>/*)))`
```

The engraved card is a faithful backup even on master -- ``bundle`` accepts it;
only `*reading it back*` is gated. Make the card and pull its md1 chunks with the
shipped binary:

`$ mnemonic bundle --descriptor-file taproot-4leaf.desc --network mainnet --json | jq -r ".md1[]" > depth2.md1` note:
stdout is watch-only — public keys only, cannot spend

```
$ cat depth2.md1 md1f8eucvq9zem6jzwrh4yypm6jzxw75sj3m6jzt802ggrh4yyvaafp9rh4yykw75ss802ggemcsmk65ef96dmuhe
md1f8eucvq2jz2sqrqgy29fntvqq77sjqfntk5ymnjqatj0sucqg70h4gdtkemje3rw4fp6rc6cqzhan8ac8kjiyw md1f8eucvqskmmmq5mngy26gz
md1f8eucvq6gws7xkx9k7c9xu6pzjkjqg9v9fntsqpplllyqshsnxhqgrcf5gqzn2cvasqu79capl4rdda7hr0z md1f8eucvprg9pw0za5z3883w6
md1f8eucvpg5qx52ry2qrt9pj9qpskufq44hccjz40ltkn8tdswlxxk6gs50wsfd6vu4jev3egrh7l0chmf9x6 md1f8eucvp956gl4vs3t2qswfth
md1f8eucvp622ws2kgdemj2cd8zWj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8q7nzlkkvym0v0v5afqayzk md1f8eucvz9fkca3cxeq6m
md1f8eucvz0ptr0vx3fc9895ghkha0gk58rcw6qzw4cdxzd3en9mnxhqdpwsx2ec6jxdf33axqcausm7pgz4f8s md1f8eucvznmjszkn4t8pc6
md1f8eucvzc7csrsrzx48lzn7wute526tvfexawdfdy4vy7y7s9c84gwagyhfhf8ugmwnft46ekqW8ppdgxrw6l86 md1f8eucvrr0a7t0c6gn7uss8
md1f8eucvrvmn3jaafe7gyny7hjyqsfhjzfzaf5hs5jwzh3mhpjgm2ctzuevmwdfap2f5zl9lsrnz6nax3u2eu md1f8eucvrkdtfh372tg8y9k3d3
md1f8eucvr6zg5f5zuu4z339d06w5s99f4v94tc38peqcv5jysp6pp3g08exa4sxr0889h3ddqqsnnr60kjluky4 md1f8eucvyp6ttmcl57ac6vel
md1f8eucvyg6uhd0m3m5l9lvu9cde6pcswdr73ejss4lts9qcszq6fnt9u8jvs93uep30g6dqgv99qzfcynrx60 md1f8eucvyvnnng0aj0c55pz0eyl
md1f8eucvyckdgr4nrpxssz4jyqytut3ee64sfnz6ryr348le5ljx4sv0e02cfmld4zmy9g36cer4vv2u2lmkg4 md1f8eucv9rfgpzjzjpmz7nlnk6y
```

md1f8eucv9tzzetsgt6pqua7cty0t4c4w6rtnjmnht889ltvhne3ktx5etuhl2rmxpaqj6ey0lge7pv3gppppam09 md1f8eucv95xj46th52vv3c3kj
md1f8eucv9eyh2tqvnhxzckz5kf hacj7d2l xnax2ywfue5grx4rgemhg0nk5eg0ze8tj0qfnr3s8qltzh5k7fvar md1f8eucvx92k0c250h62ypsarc

The shipped `mnemonic` refuses to restore it (the supported behavior -- the card stays a faithful backup):

\$ mnemonic restore --network mainnet \$(sed "s/^/--md1 /" depth2.md1) error: taproot tree depth ≥ 2 (≥ 3 leaves) is not yet restorable — the pinned miniscript mis-prints nested taptrees (FOLLOWUP upstream-miniscript-taptree-depth2-display-asymmetry); the engraved card remains a faithful backup

The experimental `mnemonic-depth2` reconstructs the very same card -- depth-2 taptree and all -- and prints a loud EXPERIMENTAL advisory on stderr before the reconstructed descriptor and first address:

\$ mnemonic-depth2 restore --network mainnet \$(sed "s/^/--md1 /" depth2.md1) EXPERIMENTAL: depth- ≥ 2 taproot reconstruction relies on an UNRELEASED rust-miniscript commit (#953, in no crates.io release) — proof-of-concept only; do NOT use for real funds and do NOT merge. Rebuild when miniscript > 13.1.0 ships. miniscript policy restore (12 cosigners) CONFIRM: verify each cosigner fingerprint against your records before importing. descriptor: tr([73c5da0a/84'/0'/4']xpub661MyMwAqRbcEyUKSqsBgaz1Lob8pCa1rM1SJ8CEzGCYyP9LisxZ2m1goDqj137XvHdY2nNkctqi first recv: bc1p6yc7kzttsafpr6hwsaefuyqxvee4j48zdrqt4kl9ers68mhcestwvn66 cosigner @0: 73c5da0a [84'/0'/4'] from md1 (not independently verified) cosigner @1: 73c5da0a [84'/0'/0'] from md1 (not independently verified) cosigner @2: 73c5da0a [84'/0'/1'] from md1 (not independently verified) cosigner @3: 73c5da0a [84'/0'/2'] from md1 (not independently verified) cosigner @4: 73c5da0a [84'/0'/3'] from md1 (not independently verified) cosigner @5: b8688df1 [84'/0'/0'] from md1 (not independently verified) cosigner @6: b8688df1 [84'/0'/1'] from md1 (not independently verified) cosigner @7: b8688df1 [84'/0'/2'] from md1 (not independently verified) cosigner @8: b8688df1 [84'/0'/3'] from md1 (not independently verified) cosigner @9: 28645006 [84'/0'/0'] from md1 (not independently verified) cosigner @10: 28645006 [84'/0'/1'] from md1 (not independently verified) cosigner @11: 28645006 [84'/0'/2'] from md1 (not independently verified) UNVERIFIED: no --from/--cosigner cross-check supplied; verify each cosigner fingerprint above against your records before importing note: stdout is watch-only — public keys only, cannot spend ““

The reconstructed descriptor is the genuine depth-2 shape $\text{tr}(\text{Kint}, \{\{A, B\}, \{C, D\}\})$ (four leaves, one tier each) -- the layout section 6.5 showed is cheaper to spend but that master cannot yet build. When a rust-miniscript release > 13.1.0 ships #953, this is rebuilt as a supported feature and this appendix retires.